



DSS Professional Bridge Development and Docking Manual



Contents

1. Overview	1
1.1 Revision History	1
1.2 Functions	1
1.3 Architecture	2
1.4 Applicable Scenarios	2
2. Event Synchronization	4
2.1 Overall Design	4
2.2 Process Design	5
2.2.1 Authorization Authentication	5
2.2.2 Event Push	11
2.3 Configuration Process	13
2.3.1 Adding Project	13
2.3.2 Synchronizing Event Types and Event Sources	14
2.3.3 Configuring Event	17
2.3.4 Deploying Bridge	18
2.3.5 Reporting Events	18
2.4 Test Process	19
3. System Integration-Access Control Platform	20
3.1 Overall Design	20
3.2 Process Design	21
3.2.1 Authorization Authentication	21
3.2.2 Event Push	21
3.2.3 Data Synchronization	22
3.2.4 Access Control Operation	25
3.2.5 Access Control Records	27
3.2.6 Device Status	29
3.3 Configuration Process	30
3.3.1 Adding Project	30
3.3.2 Synchronizing Event Types	31
3.3.3 Deploying Bridge	31
3.3.4 Synchronizing Data	31
3.3.5 Configuring Event	33
3.3.6 Reporting Events	35
3.3.7 Access Control Operations	35
3.3.8 Access Control Records	36
3.4 Test Process	37
4. Description of Related API Interfaces	40
4.1 Authorization Authentication	40
4.2 Event Push	43
4.3 Synchronizing Data	46
4.4 Access Control Operation	50
4.5 Access Control Records	52

4.6 Device Status	54
5. FAQ.....	55

Dahua-Bridge-Development-and-Docking-Manual_V1.1.1_20240715 for HOTEL_BUDING

1. Overview

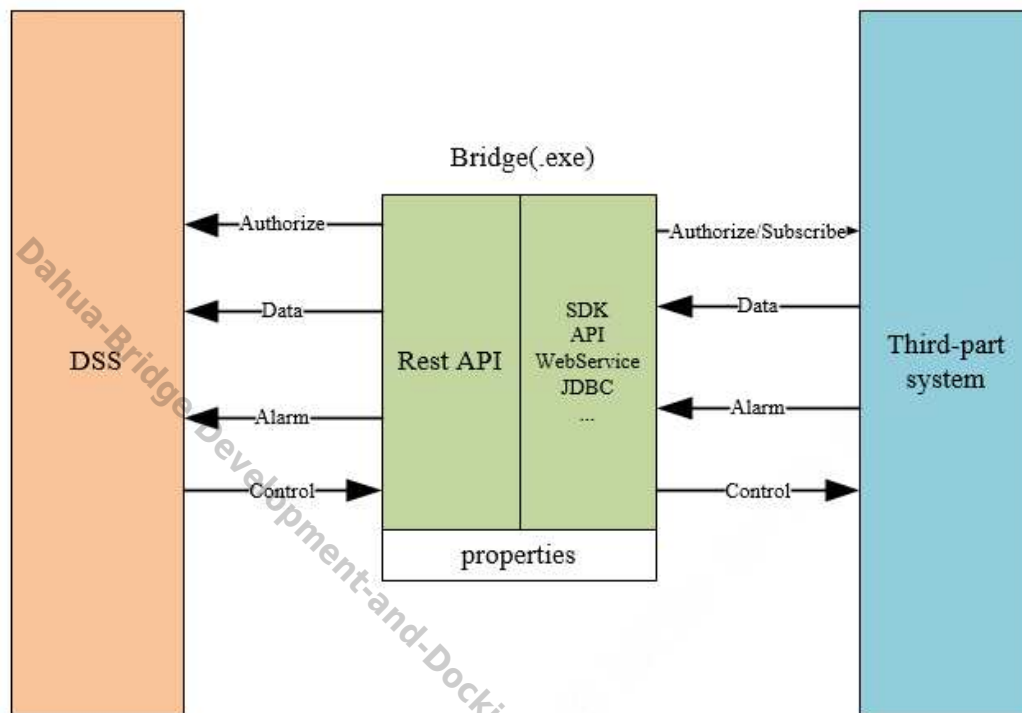
1.1 Revision History

Document Version	Software Version	Revision Content	Release Time
V1.0.0	V8.000.00000004.0 V8.001.00000000.0 V8.002.00000000.0 V8.003.00000000.0	First release.	December 2021
V1.1.0	V8.004.00000000.0	Added system integration-access control platforms.	January 2024
V1.1.1	V8.005.00000000.0	Event synchronization, added image and video resource acquisition.	July 2024

1.2 Functions

We defined a message transfer station called the Bridge to enable real-time communication of data and events between DSS platform and third-party systems, and the basic reverse control commands by DSS platform on a third-party platform. The Bridge allows for a smooth exchange of data and events between different systems.

1.3 Architecture



As shown in the figure above, the Bridge interacts with DSS platform and third-party systems. Interaction with DSS platform follows a standardized process, while the interaction with third-party systems is tailored to meet the specific requirements of each system. Consequently, it is essential to customize the interaction between each third-party system and the Bridge to ensure accurate transmission of data and events to DSS platform.

The Bridge supports the following functions:

1. Third-party event reporting and event linkage.
2. Supports asynchronous and synchronous reporting operations for third-party events that include images and videos.
3. Integration of third-party access control systems.
 - 1) Opening and closing doors.
 - 2) Synchronization of access control devices and person information.
 - 3) Extraction and synchronization of access control records.
 - 4) Real-time offline status of devices and door status.

1.4 Applicable Scenarios

The Bridge can be used to integrate third-party access control platforms to DSS platform and synchronize events from third-party platforms to DSS platform.

Integration: Data and events are transferred from third-party platforms to DSS platform. Additionally, it enables linkage between DSS platform and third-party platforms.

Event Synchronization: Only event, image and video linkage is supported. The data of third-party platforms DSS platform does not interact.

Users can select different functions based on their requirements.

Dahua-Bridge-Development-and-Docking-Manual_V1.1.1_20240715 for HOTEL_BUDING

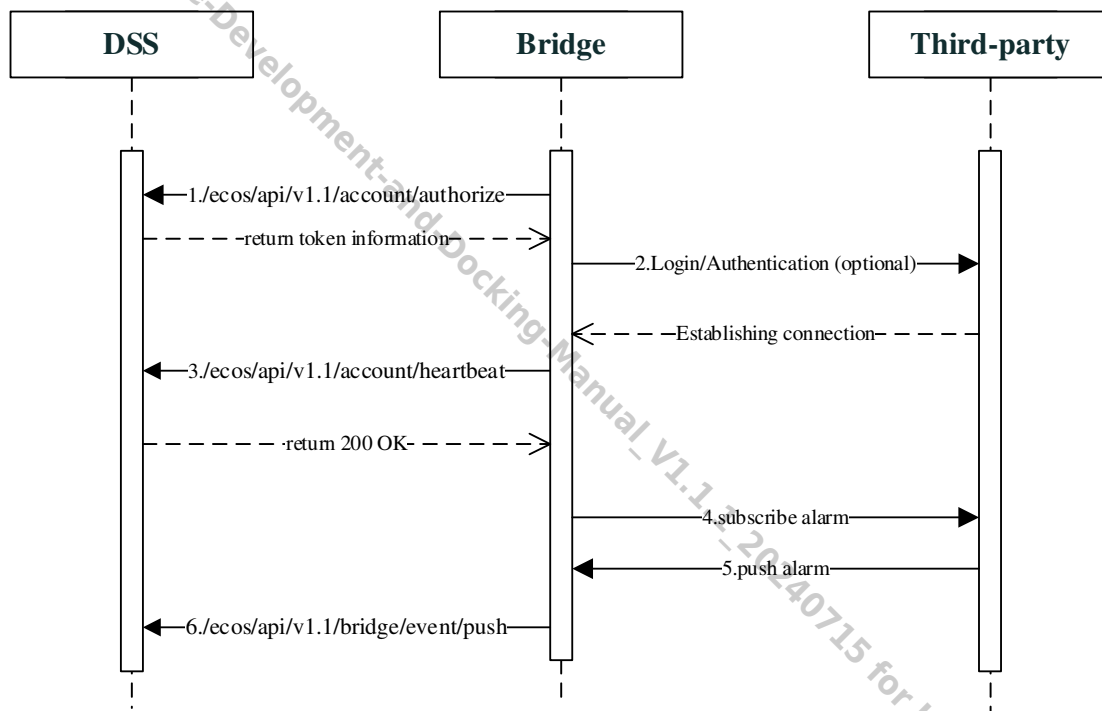
2. Event Synchronization

2.1 Overall Design

Event synchronization enables data interaction and information synchronization between third-party systems and DSS platform. This feature primarily entails two functions:

- 1) Establishing communication with the platforms through authorization authentication.
- 2) Protocol conversion, data aggregation, and secondary processing are performed on the synchronized events, ensuring that the data remains unchanged.

The following figure shows the overall work flow.

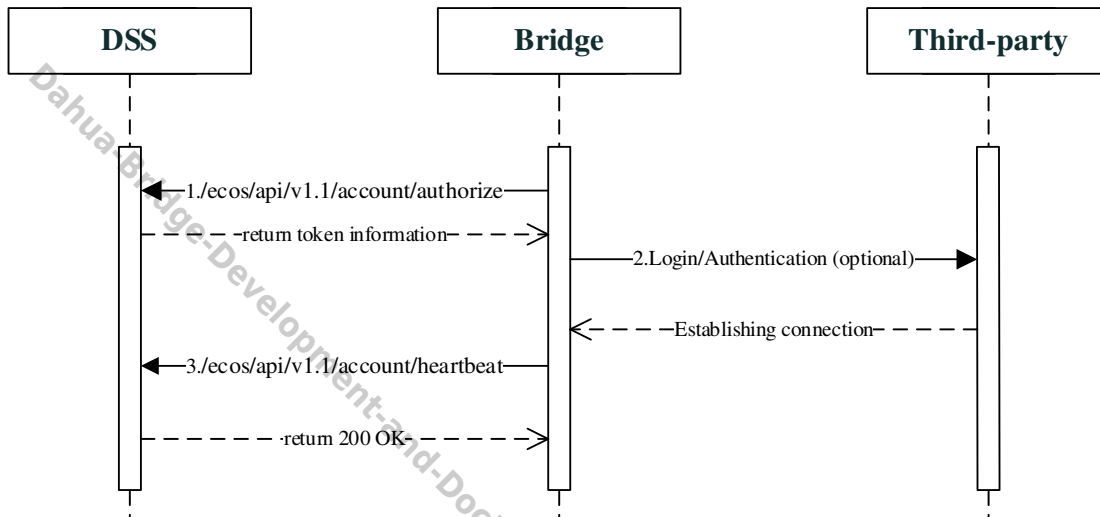


As shown in the figure, events synchronization is mainly divided into two steps: Authorization authentication and event pushing.

2.2 Process Design

2.2.1 Authorization Authentication

2.2.1.1 Workflow



1. **Authorization authentication.** The authentication method adopted is HMAC(AK/SK). The Bridge carries the agreed Access Key and the signature encrypted by Secret Key to initiate an authentication request /authorize. The signature is generated by the HMAC-SHA256 (data, key) algorithm, in which data equals Timestamp and key equals Secret Key. DSS platform uses the same algorithm to generate the signature and compares it with the signature in the request. If the signatures are the same, the verification is passed. Then, the platform automatically generates a check code token, returns it to the requester, and caches a copy of it. The validity period is 30 minutes, after which the copy will be cleared automatically. All subsequent interface calls require that the Bridge carry X-Subject-Token:{token} in the request header.
2. **Acquire the configurations.** To ensure the security of the third-party connection information, after completing the authorization and authentication of the DSS platform, the Bridge needs to obtain the configured encryption information of the third-party connection from the platform, which will be used to establish connections with the third-party systems. This is an optional interface and the connection information can also be directly set in the profile.
3. **Keep the heartbeat.** Heartbeat detection is supported. The Bridge can call the /heartbeat interface to be alive. The interface can set the Bridge to be online. This is an extended function and is not a necessary option for the authentication process. However, the Bridge can be displayed as online on the DSS client only when the heartbeat interface is called.
4. **Update Token.** Because the validity period of tokens is 30 minutes, you must call the token update interface/token/update before the token expires. The interface will send a new token, and the original token will be

retained until it automatically expires. So, the message in the critical point of token updating can be ensured not to be lost. If the token has expired, the /authorize interface should be called again.

2.2.1.2 Related APIs

Request Path Overview:

Function	URI	Request Type
Uses AK/SK mode for authentication and verification of user permissions, and issues tokens if the verification is passed.	/ecos/api/v1.1/account/authorize	HTTP POST
Obtain the configured information of third-party connection	/ecos/api/v1.1/bridge/{id}/config	HTTP GET
Update token. And the original token will automatically expire after being retained for 30 minutes. We recommend calling this interface every 25 minutes.	/ecos/api/v1.1/account/token/update	HTTP POST
Heartbeat keeps alive. We recommend calling it every 25 seconds.	/ecos/api/v1.1/account/heartbeat	HTTP POST

Note: See chapter 4 for API details.

2.2.1.3 Example of Calling API

Pom dependencies to be introduced:

```
<!-- Json -->
<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>fastjson</artifactId>
    <version>1.2.58</version>
</dependency>

<!-- Apache HTTP Client -->
<dependency>
    <groupId>org.apache.httpcomponents</groupId>
    <artifactId>httpclient</artifactId>
    <version>4.5.12</version>
</dependency>
```

Authentication example (Java version):

```
import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONObject;
import org.apache.http.client.methods.CloseableHttpResponse;
import org.apache.http.client.methods.HttpEntityEnclosingRequestBase;
import org.apache.http.client.methods.HttpPost;
import org.apache.http.entity.StringEntity;
```

```
import org.apache.http.impl.client.CloseableHttpClient;
import org.apache.http.impl.client.HttpClients;
import org.apache.http.util.EntityUtils;

import java.io.IOException;
import java.util.HashMap;
import java.util.Locale;
import java.util.Map;

/**
 * Authorize to DSS
 * Step 1: Authorize
 *   * Step 2: GetConfig
 * Step3: Heartbeat
 * Step4: Update token
 */
public class Authorize2DSS {

    // Get this information from the configuration file
    private static final String DSS_IP = "192.168.1.1";
    private static final int DSS_PORT = 80;
    private static final String ACCESS_KEY = "f12M3OfILd7XUJpopfilulw8";
    private static final String SECRET_KEY = "z9gc721UGO7p9eSi5Z40Flg=";

    public static String TOKEN_VALUE = null;
    public static final int SUCCESS_CODE = 1000;
    private static int HEART_COUNT = 0;

    public static void main(String[] args) throws Exception {
        // Step 1: Authorize
        authorize();
        getConfig();

        while (true) {
            // Step2: Heartbeat
            Map<String, Object> heartbeatParamMap = new HashMap<>(1);
            heartbeatParamMap.put("token", TOKEN_VALUE);
            String heartbeatResponseString = sendPost(buildHttpUrl("account/heartbeat"), heartbeatParamMap);
            JSONObject heartbeatResponse = JSONObject.parseObject(heartbeatResponseString);
            if (heartbeatResponse.getInteger("code").intValue() == SUCCESS_CODE) {
                HEART_COUNT++;
                // Success: Send heartbeat every 25 seconds
                Thread.sleep(25000);
            } else {
                // Fail: Re authorize
                authorize();
            }
        }
    }
}
```

```

        HEART_COUNT = 0;
    }

    // Step3: Update token
    // Update the token every 25 minutes
    if (HEART_COUNT % 50 == 0) {
        // Restore the heartbeat count to zero
        HEART_COUNT = 0;
        Map<String, Object> updateTokenParamMap = new HashMap<>(3);
        String timestamp = String.valueOf(System.currentTimeMillis());
        updateTokenParamMap.put("signature", EncryptionUtils.HMACSHA256(timestamp, SECRET_KEY));
        updateTokenParamMap.put("timestamp", timestamp);
        updateTokenParamMap.put("token", TOKEN_VALUE);
        String updateTokenResponseString = sendPost(buildHttpUrl("account/token/update"), updateTokenParamMap);
        JSONObject updateTokenResponse = JSONObject.parseObject(updateTokenResponseString);
        if (updateTokenResponse.getInteger("code").intValue() == SUCCESS_CODE) {
            // Success: Reset token
            TOKEN_VALUE = updateTokenResponse.getJSONObject("data").getString("token");
        } else {
            // Fail: Re authenticate
            authorize();
            HEART_COUNT = 0;
        }
    }
}

}

/**
 * Authorize to DSS
 * @throws Exception
 */
private static void authorize() throws Exception {
    Map<String, Object> authorizeParams = new HashMap<>(3);
    String timestamp = String.valueOf(System.currentTimeMillis());
    authorizeParams.put("accessKey", ACCESS_KEY);
    authorizeParams.put("signature", EncryptionUtils.HMACSHA256(timestamp, SECRET_KEY));
    authorizeParams.put("timestamp", timestamp);
    String authorizeResponseString = sendPost(buildHttpUrl("account/authorize"), authorizeParams);
    JSONObject authorizeResponse = JSONObject.parseObject(authorizeResponseString);
    TOKEN_VALUE = authorizeResponse.getJSONObject("data").getString("token");
}

/**
 * GetConfig from DSS
 * @throws Exception
 */

```

```
private static void getConfig() throws Exception {
    Map<String, Object> header = new HashMap<>();
    header.put("Accept", "application/json, text/plain, */*");
    header.put("Connection", "keep-alive");
    header.put("Accept-Language", "en");
    header.put("X-Subject-Token", "token");
    String configResponseString = sendGet(buildHttpUrl("ecos/api/v1.1/bridge/{bridgedId}/config"), header);
    JSONObject configResponse = JSONObject.parseObject(configResponseString);
}
```

```
/**
```

```
 * Send post request method with HTTP
```

```
 * @param url
```

```
 * @param params
```

```
 * @return reply
```

```
 */
```

```
private static String sendPost(String url, Map<String, Object> params) throws IOException {
```

```
    CloseableHttpClient httpClient = HttpClients.createDefault();
```

```
    HttpEntityEnclosingRequestBase httpRequest = new HttpPost(url);
```

```
    StringEntity entity = new StringEntity(JSON.toJSONString(params), "UTF-8");
```

```
    httpRequest.setEntity(entity);
```

```
    httpRequest.setHeader("Accept-Language", Locale.getDefault().getLanguage());
```

```
    httpRequest.setHeader("Content-Type", "application/json;charset=UTF-8");
```

```
    if (params.containsKey("token")) {
```

```
        httpRequest.setHeader("X-Subject-Token", params.get("token").toString());
```

```
    }
```

```
    CloseableHttpResponse response = httpClient.execute(httpRequest);
```

```
    //In order to avoid messy code, encode the response data in UTF-8.
```

```
    String reply = EntityUtils.toString(response.getEntity(), "UTF-8");
```

```
    //Release resources finally.
```

```
    if (httpClient != null) {
```

```
        httpClient.close();
```

```
    }
```

```
    if (response != null) {
```

```
        response.close();
```

```
    }
```

```
    return reply;
```

```
}
```

```
/**
```

```
 * Send post request method with HTTP
```

```
 * @param url
```

```
 * @param params
```

```
 * @return reply
```

```
 */
```

```
private static String sendPut(String url, Map<String, Object> params) throws IOException {
```

```

        CloseableHttpClient httpClient = HttpClients.createDefault();
        HttpEntityEnclosingRequestBase httpRequest = new HttpPut(url);
        StringEntity entity = new StringEntity(JSON.toJSONString(params), "UTF-8");
        httpRequest.setEntity(entity);
        httpRequest.setHeader("Accept-Language", Locale.getDefault().getLanguage());
        httpRequest.setHeader("Content-Type", "application/json;charset=UTF-8");
        if (params.containsKey("token")) {
            httpRequest.setHeader("X-Subject-Token", params.get("token").toString());
        }
        CloseableHttpResponse response = httpClient.execute(httpRequest);
        //In order to avoid messy code,encode the response data in UTF-8.
        String reply = EntityUtils.toString(response.getEntity(), "UTF-8");
        //Release resources finally.
        if (httpClient != null) {
            httpClient.close();
        }
        if (response != null) {
            response.close();
        }
        return reply;
    }

```

```
/**
```

```
 * Build http url
```

```
 * @param uri uri
```

```
 * @return Http url
```

```
 */
```

```
private static String buildHttpUrl(String uri) {
```

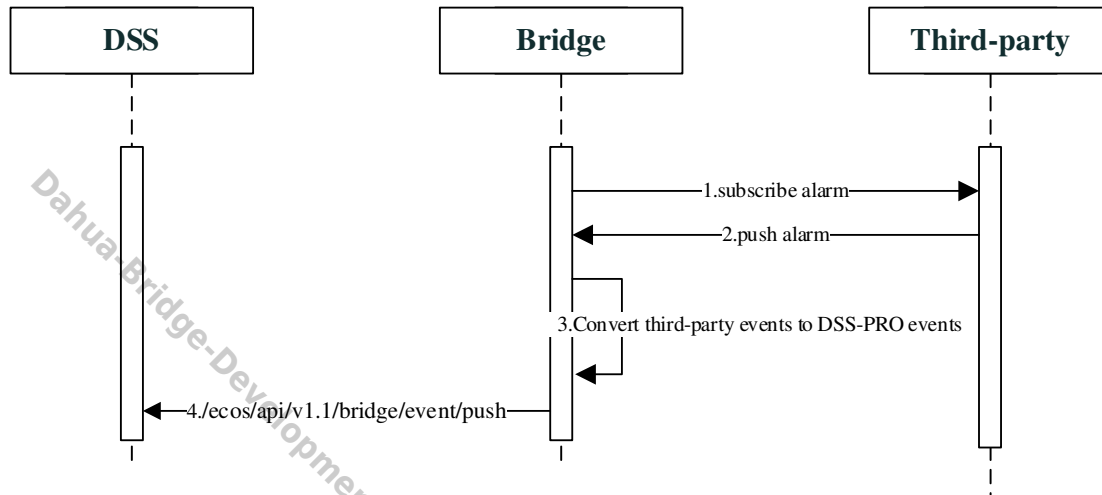
```
    return String.format("http://%s:%s/ecos/api/v1.1/%s", DSS_IP, DSS_PORT, uri);
```

```
}
```

```
}
```

2.2.2 Event Push

2.2.2.1 Workflow



1. Receive event messages from third-party systems. It involves interaction with a third-party system. Make sure that the third-party system can successfully transmit event messages to the Bridge.

2. Convert protocols. It involves converting the protocol employed by the third-party system into a format that is compatible with DSS platform. This guarantees the data accuracy and consistency. Do not modify the data. During protocol conversion, you need to ensure the integrity and accuracy of the data and do not modify the data.

3. Push events. Call the /event/push API to push to DSS platform. It involves interaction with DSS platform. Make sure that Bridge can correctly send event messages to DSS platform.

4. Push files. Call the /event/file/push/batch interface. Push the images and videos carried by the third-party event to the platform for display.

5. Print logs. We recommend incorporating log level control into your codes. By appropriately implementing log recording, you can effectively troubleshoot issues. We recommend including the log recording function in the code for event push and setting different log levels as needed. For example, you can set the debug level during debugging and the error level during operation. This approach simplifies the troubleshooting process, reduces unnecessary log output, and enhances system throughput.

Note: To handle high concurrency, we have introduced the /event/push/batch API for pushing events in batches. This API is particularly useful when a significant volume of events need to be pushed by a third-party system. By calling this API at regular intervals, events can be accurately and promptly synchronized in real-time.

2.2.2.2 Related APIs

Request Path Overview:

Function	URI	Request Type
Push event messages	/ecos/api/v1.1/bridge/event/push	HTTP POST
Push event images and videos	/ecos/api/v1.1/bridge/event/file/push/batch	HTTP POST

Note: See chapter 4 for API details.

2.2.2.3 Example of Calling API

Example of pushing events (Java version):

```
/**
 * Step4: Send event(The focus is to show ideas)
 */
public static void putEvent() throws Exception {
    // 1.Get third-part system event message from queue
    BlockingQueue<Object> eventQueue = new LinkedBlockingQueue<Object>(5000);
    List<Object> handleEventList = new ArrayList<Object>();
    eventQueue.drainTo(handleEventList, 10);

    // 2.Convert third-part system message to DSS message
    for (int i=0; i<handleEventList.size(); i++) {
        Map<String, Object> putEventParams = new HashMap<>(7);
        putEventParams.put("eventId", "{C6FF341F-2BoD-425B-B788-39E9E2606AE9}");
        putEventParams.put("eventTime", 1615248000);
        putEventParams.put("eventSourceCode", "1000005$1$0$2");
        putEventParams.put("eventSourceName", "Entrance Door");
        putEventParams.put("eventTypeCode", "51");
        putEventParams.put("eventTypeName", "Valid Swipe");
        putEventParams.put("eventSourceStatus", "1");
        putEventParams.put("pictureCount", "2");
        putEventParams.put("videoCount", "3");
        putEventParams.put("remark", "This is an event");

        // 3.Send message to DSS system
        sendPost(buildHttpRequest("bridge/event/push"), putEventParams);
    }
}
```

Example of pushing files in batches (Java version):

```
/**
 * Step5: Push files in batches
 */
public static void batchPutEventFile() throws Exception {
    List<Map<String, Object>> list = new ArrayList<>();
    for (int i=0; i<handleEventFileList.size(); i++) {
        Map<String, Object> events = new HashMap<>(7);
        events.put("fileUrl", "http://file.jpg");
        events.put("fileType", 1);
        events.put("recordTime", 1735353576);
        events.put("fileTimeLength", 2314);

        list.add(events);
    }
}
```

```

Map<String, Object> putEventParams = new HashMap<>(1);
putEventParams.put("files",list);
putEventParams.put("associateId", "rgwegrdfzgxwergw");
// 3.Send message to DSS system
sendPost(buildHttpUrl("bridge/event/file/push/batch"), putEventParams);
}

```

Example of pushing events in batches (Java version):

```

/**
 * Step5: Push events in batches
 */
public static void batchPutEvent() throws Exception {
    // 1.Get third-part system event message from queue
    BlockingQueue<Object> eventQueue = new LinkedBlockingQueue<Object>(5000);
    List<Object> handleEventList = new ArrayList<Object>();
    eventQueue.drainTo(handleEventList, 10);

    // 2.Convert third-part system message to DSS message
    List<Map<String, Object>> list = new ArrayList<>();
    for (int i=0; i<handleEventList.size(); i++) {
        Map<String, Object> events = new HashMap<>(7);
        events.put("eventId", "{C6FF341F-2BoD-425B-B788-39E9E2606AE9}");
        events.put("eventTime", 1615248000);
        events.put("eventSourceCode", "1000005$1$0$2");
        events.put("eventSourceName", "Entrance Door");
        events.put("eventTypeCode", "51");
        events.put("eventTypeName", "Valid Swipe");
        events.put("remark", "This is an event");

        list.add(events);
    }
    Map<String, Object> putEventParams = new HashMap<>(1);
    putEventParams.put("events",list);
    // 3.Send message to DSS system
    sendPost(buildHttpUrl("bridge/event/push/batch"), putEventParams);
}

```

2.3 Configuration Process

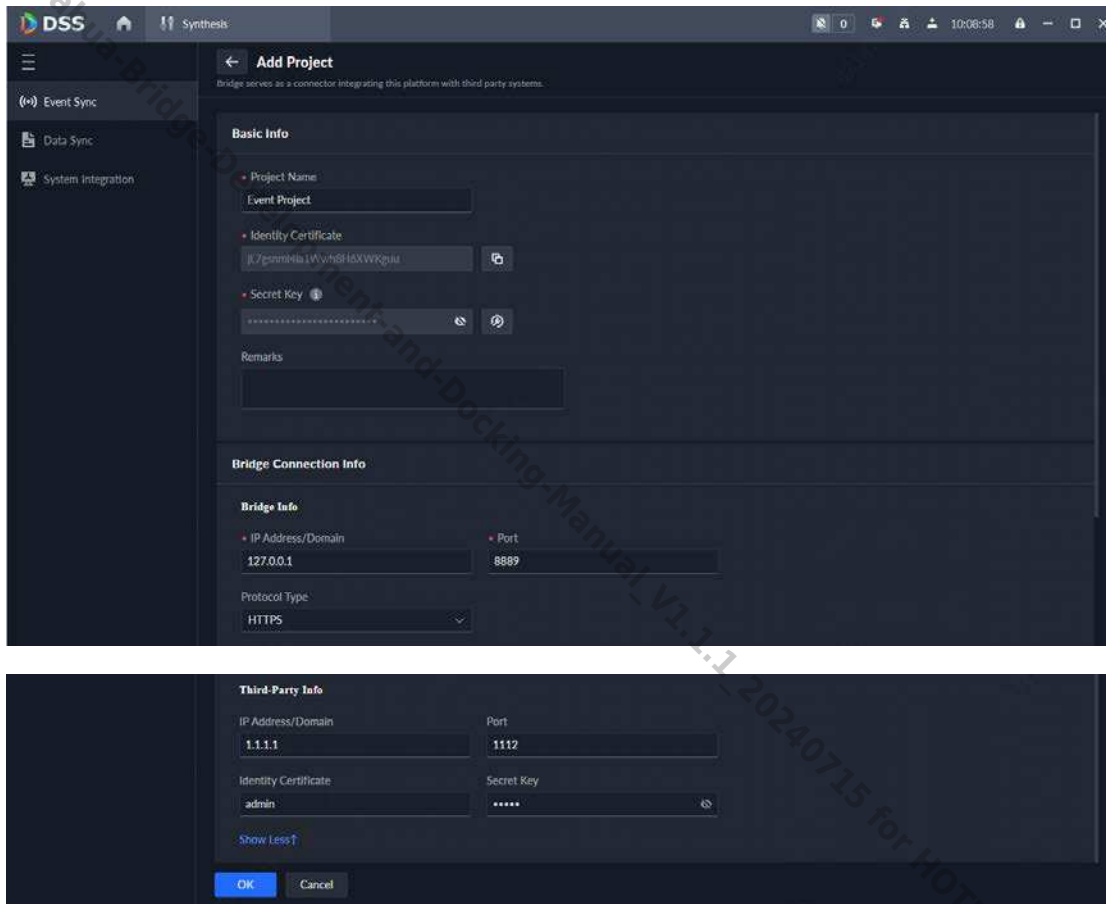
To complete the entire process and validate the accuracy of the functions, you need to add a Project on the DSS client and configure the event type and event source information.

2.3.1 Adding Project

Select  > Synthesis > Event Sync > Add Now.

Enter the project name. To enhance security, the platform automatically generates a pair of identity certificate and secret key. These keys can be reset if needed. To view and reset the keys, you need to enter the login password. Additionally, both fields can be copied for convenient use in future maintenance of the Bridge configuration file. The subsequent authentication of the Bridge to the platform will require these specific credentials and keys to be carried.

Enter the IP address and port of the Bridge. Select the protocol type for communication between the Bridge and the platform. It is HTTPS by default. The Bridge needs to follow this protocol for interaction with the platform, otherwise it will not be able to go online. Fill in the connection information between the Bridge and the third-party system (optional). Click **OK** to save the project information.



2.3.2 Synchronizing Event Types and Event Sources

Once the project is added, it is necessary to add the third-party event types and event sources to be reported to DSS platform.

You can add event types one by one or in batches. Once added, you can view the event types on the project page.

Incoming Triggered Event Type

+ Add
Delete
Import
Search...

☐	Incoming Triggered Event Type Code	Incoming Triggered Event Type Name	Operate
☐	event-code	event name	

Total 1 Record(s)
1
20/Page

Edit the source status of the event (optional), configure the input source status information. Supports users to view the status information of the enter event source corresponding to the bridge, or use this data to extend more applications in the future. The status can represent the offline situation of the event source, or the health status or real-time status (such as alarms) of the event source. Here, only the configuration and data entry are provided, and the specific application depends on the apply. The event source supports multiple statuses at the same time, and the combination mode can be enabled to set them. The call `/event/source/status/reset` interface can reset the status of some or all event sources in the bridge.

▼ Status of Incoming Event Source

Composite Mode
☐

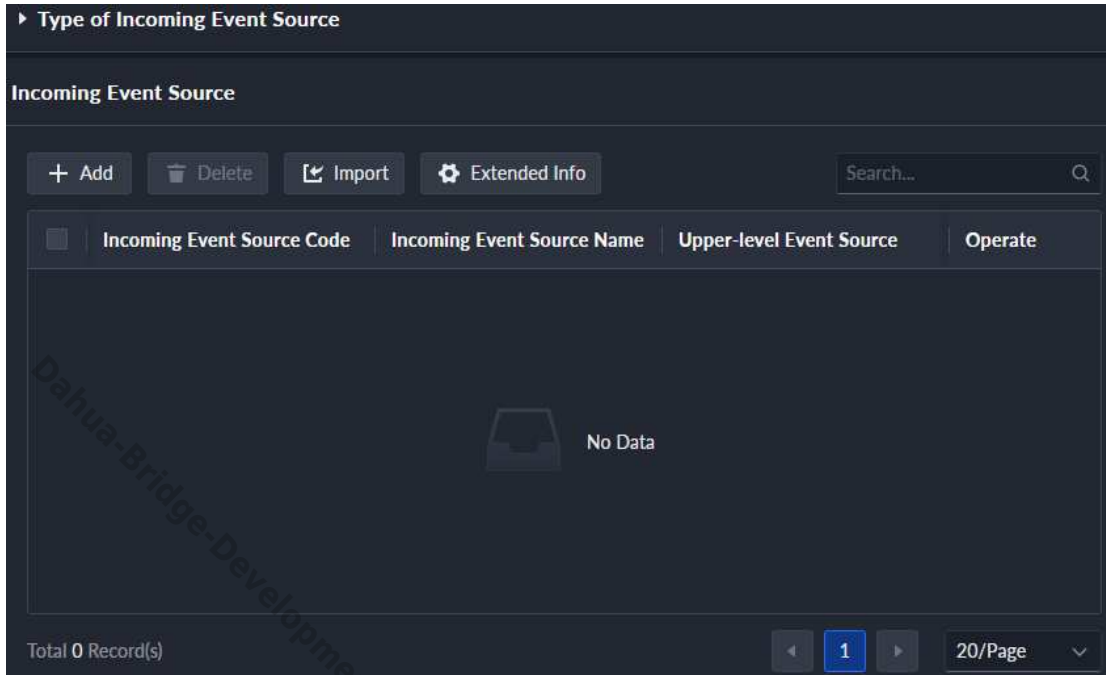
If an event source has multiple statuses, the platform will combine and display the statuses based on the status of the event source that was added when information was received on it.

+ Add
Delete
Import
Search...

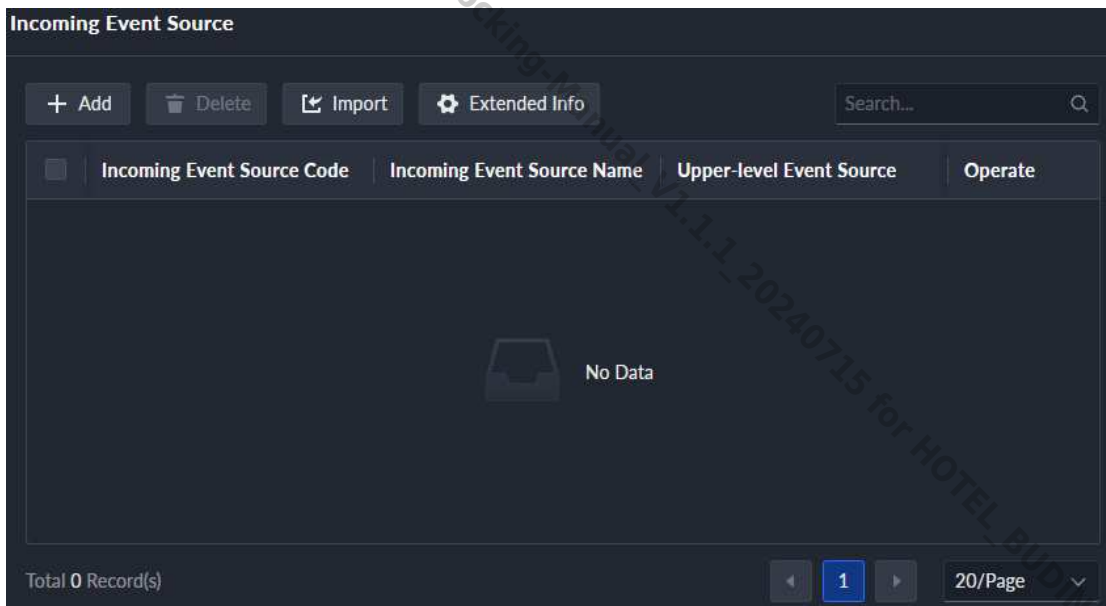
☐	Status Code of Incoming Event Source	Status Name of Incoming Event Source	Operate
☐	110	online	

Total 1 Record(s)
1
20/Page

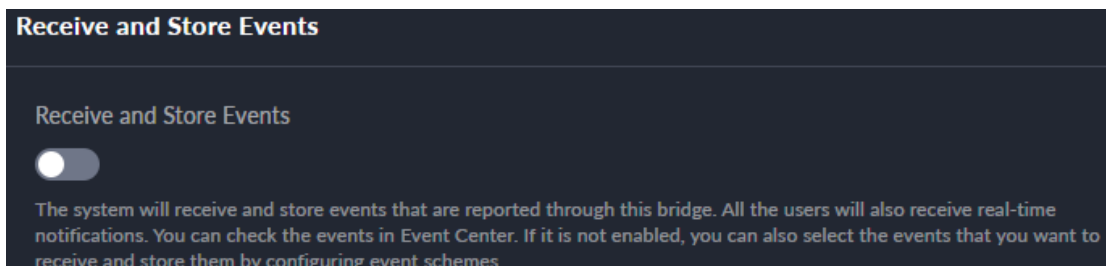
Edit event source type (optional), configure input source type information, support users to view the type information of the enter event source when viewing the Bridge, or future applications can use this data to extend more applications. The event source type is a basic attribute of the event source, and once configured, it will not change. There is no need to push this information in the event later.



You can add event sources one by one or in batches. Once added, you can view the event source on the Project page.

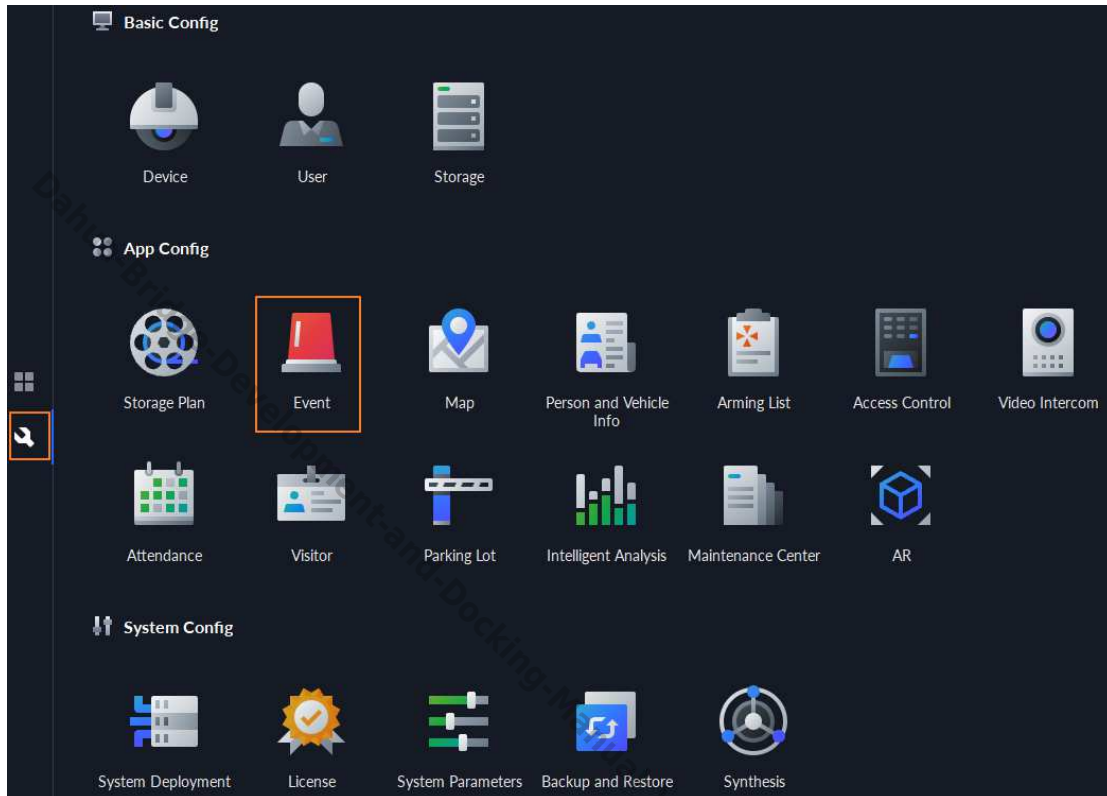


Configure the event to receive and store (optional), and configure whether the Bridge corresponding to the event directly stores and notifies all client users in real time. However, if the linkage function is required, you have to configure the scheme in the event center.

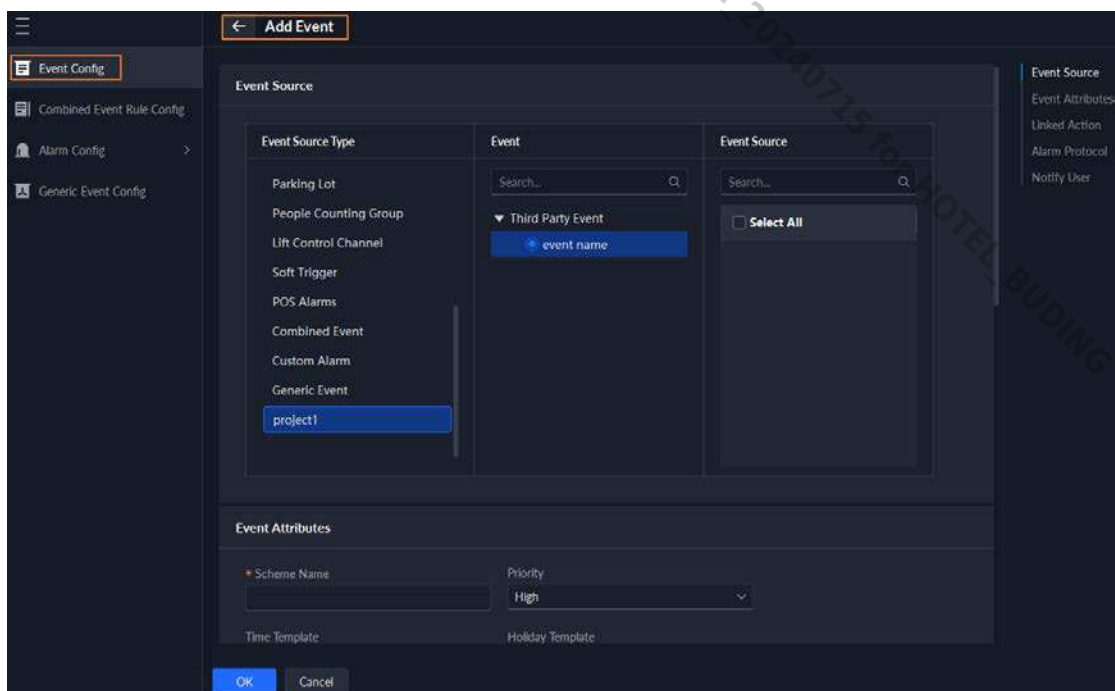


2.3.3 Configuring Event

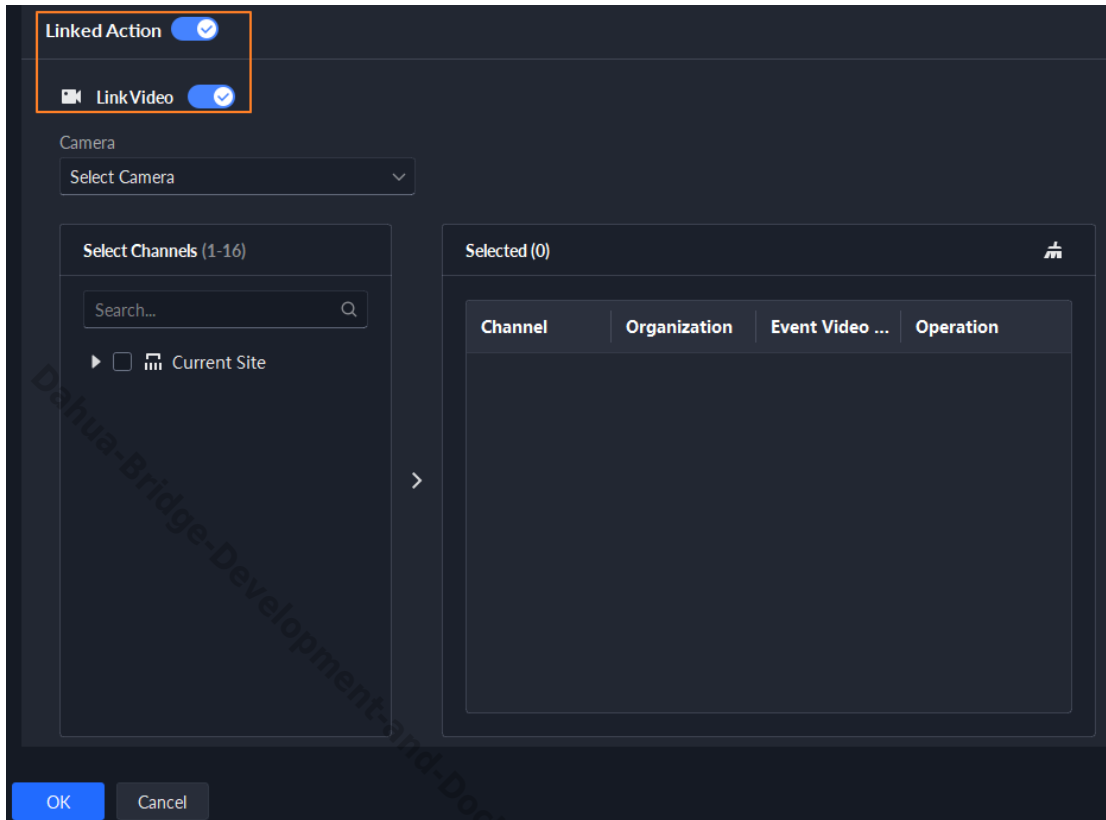
Select  > Event > Event Config > Add.



Select the event type and event source.



Enter the event name. Enable **Linked Action** and **Linked Video**. Select the video to be linked.



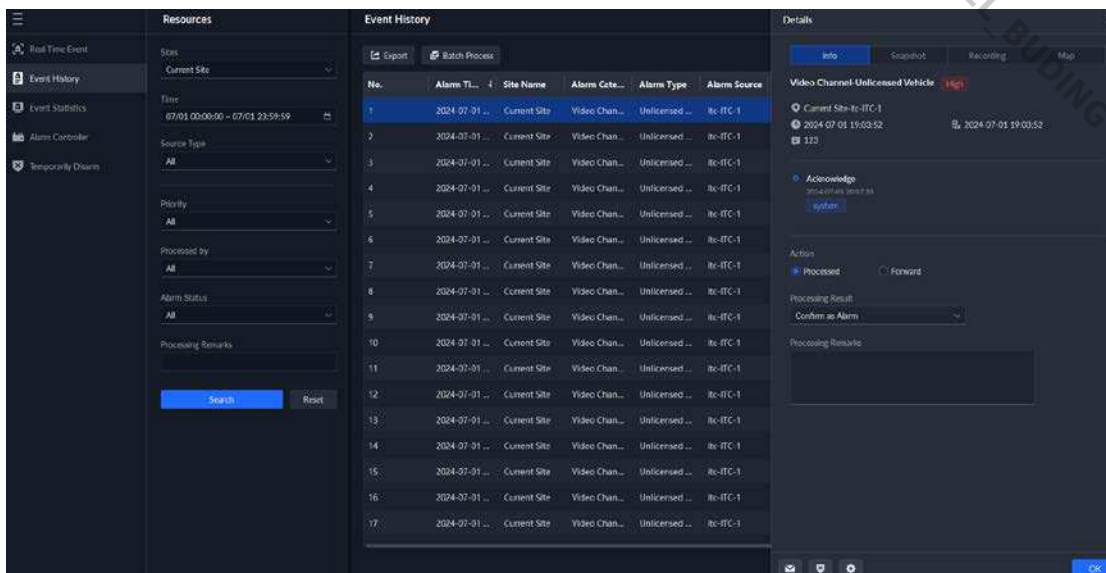
Click **OK** to add the event.

2.3.4 Deploying Bridge

Deploy the Bridge on the designated server based on your own logic, ensuring the access between the bridge, DSS platform and third-party platforms.

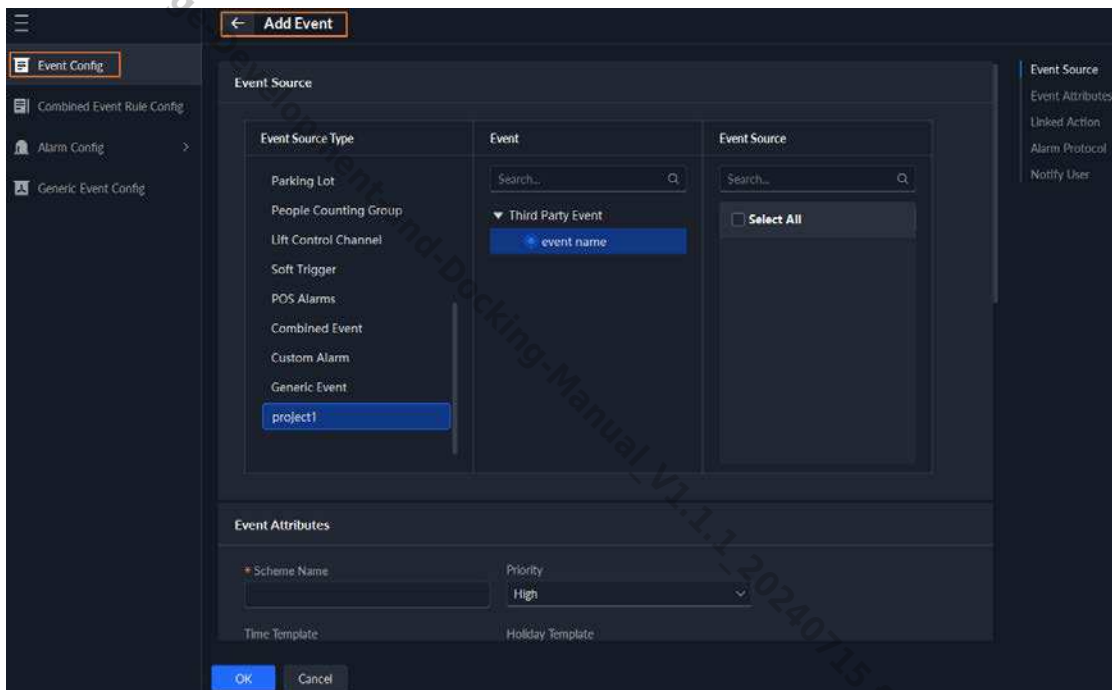
2.3.5 Reporting Events

When an event arises on a third-party platform, it is pushed to DSS platform in real time. DSS platform determines whether the event is reported on the platform based on the event type and event source configured in the event synchronization, and performs video linkage. You can view subsequent events in the **Event History**.



2.4 Test Process

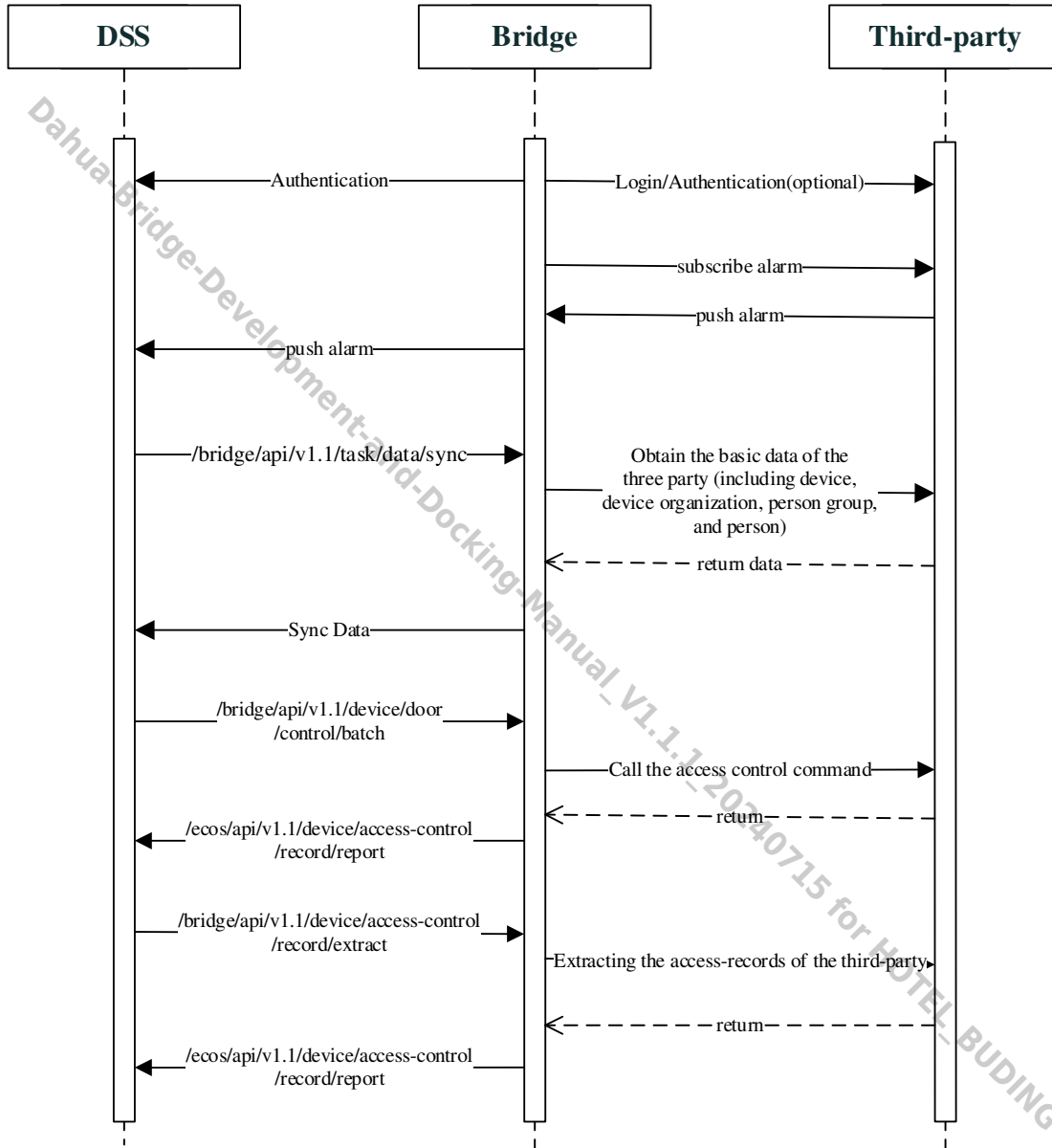
- 1) Add a Bridge on the DSS client and configure the information of event synchronization.
- 2) Deploy the Bridge and configure the connection information between DSS platform and third-party systems, such as IP, Port, Access Key, Secret Key, Username, and Password. Ensure that both systems can be successfully logged in.
- 3) Select  > **App Config** > **Event** on the DSS client and select the configured Bridge to add an event scheme. You should configure the number event schemes based on the number of Triggered Events, as shown in the figure below. When configuring **Linked Action** according to business needs, you must select one or more users to be notified. The users will receive the event.



- 4) Trigger events in third-party systems.
- 5) Select  > **Event Center** > **Real-time Event** on the DSS client to check whether the third-party events are received. If they are received, the test is complete, otherwise, follow the FAQ for troubleshooting.

3. System Integration-Access Control Platform

3.1 Overall Design



System integration supports the following functions:

1. Synchronization of basic data related to access control.
2. Opening and closing doors.
3. Report of access control events and alarms.
4. Extraction and synchronization of access control records.
5. Synchronization of offline status of devices and door status.

3.2 Process Design

3.2.1 Authorization Authentication

3.2.1.1 Workflow

See section 2.2.1.1.

3.2.1.2 Related APIs

See section 2.2.1.2.

3.2.1.3 Example of Calling API

See section 2.2.1.3.

3.2.2 Event Push

3.2.2.1 Workflow

See section 2.2.2.1.

3.2.2.2 Related APIs

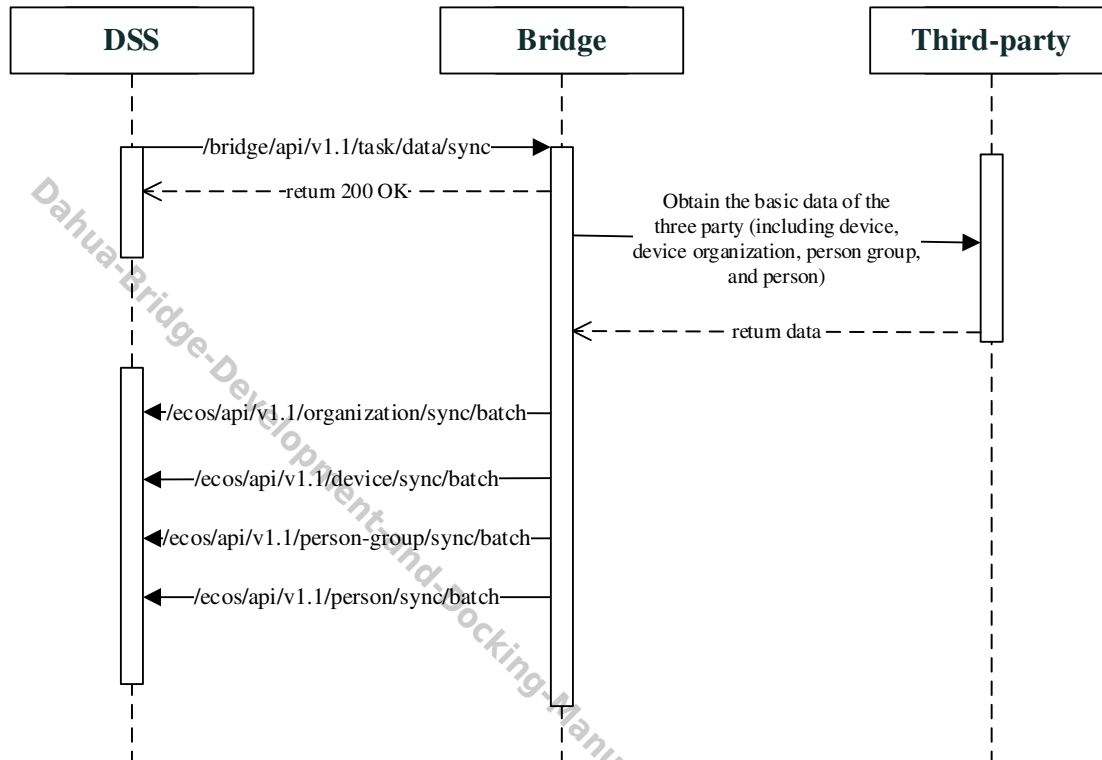
See section 2.2.2.2.

3.2.2.3 Example of Calling API

See section 2.2.2.3.

3.2.3 Data Synchronization

3.2.3.1 Workflow



DSS platform supports synchronization of the third-party basic data, such as device organization, device, person group, and persons. You can manually synchronize the basic data or configure automatic synchronization. By default, the Bridge needs to provide a `/bridge/api/v1.1/task/data/sync` interface to synchronize third-party data. DSS platform actively calls this interface to acquire the third-party basic data. Once the basic data is obtained, you can call the synchronization interface of the device organization, device, person group, and person to synchronize the data to DSS platform.

Note that even if a certain type of basic data is empty, you still need to call the corresponding synchronization interface. If the request parameter is empty, you need to call all 4 synchronization interfaces simultaneously. The synchronization task will then be concluded.

3.2.3.2 Related APIs

Request Path Overview:

Function	URI	Request Type
Perform third-party platform data synchronization tasks (Bridge needs to provide)	<code>/bridge/api/v1.1/task/data/sync</code>	HTTP POST
Synchronize device organizations.	<code>/ecos/api/v1.1/organization/sync/batch</code>	HTTP POST
Synchronize devices.	<code>/ecos/api/v1.1/device/sync/batch</code>	HTTP PUT
Synchronize person groups.	<code>/ecos/api/v1.1/person-group/sync/batch</code>	HTTP PUT
Synchronize people.	<code>/ecos/api/v1.1/person/sync/batch</code>	HTTP PUT

Note: See chapter 4 for API details.

3.2.3.3 Example of Calling API

Example of synchronizing device organizations (Java version):

```
/**
 * Synchronize device organizations in batches
 * @throws Exception
 */
public static void syncDeviceOrganizations() throws Exception {
    // 1. Obtain the third-party device organizations
    BlockingQueue<Object> eventQueue = new LinkedBlockingQueue<Object>(5000);
    List<Object> handleEventList = new ArrayList<Object>();
    eventQueue.drainTo(handleEventList, 10);

    // 2. Convert the third-party device organizations to DSS platform format
    List<Map<String, Object>> list = new ArrayList<>();
    for (int i=0; i<handleEventList.size(); i++) {
        Map<String, Object> organizations = new HashMap<>(7);
        organizations.put("orgName", "DoorGroup1");
        organizations.put("parentOrgCode", "");
        organizations.put("orgCode", "4");
        organizations.put("remark", "remark");

        list.add(organizations);
    }
    Map<String, Object> postParams = new HashMap<>(1);
    postParams.put("organizations", list);
    // 3. Push the data of device organizations to DSS platform
    sendPost(buildHttpUrl("organization/sync/batch"), postParams);
}
```

Example of synchronizing devices (Java version):

```
/**
 * Synchronize devices in batches
 * @throws Exception
 */
public static void syncDevices() throws Exception {
    // 1. Obtain the third-party devices
    BlockingQueue<Object> eventQueue = new LinkedBlockingQueue<Object>(5000);
    List<Object> handleEventList = new ArrayList<Object>();
    eventQueue.drainTo(handleEventList, 10);

    // 2. Convert the third-party devices to DSS platform format
    List<Map<String, Object>> list = new ArrayList<>();
    for (int i=0; i<handleEventList.size(); i++) {
        Map<String, Object> devices = new HashMap<>(7);
        devices.put("addType", "4");
    }
}
```

```

        devices.put("deviceIp", "-");
        devices.put("deviceName", "deviceName");
        devices.put("devicePort", "37777");
        devices.put("deviceCategory", "8");
        devices.put("deviceType", "99");
        devices.put("associateId", "12345");
        devices.put("registerId", UUID.randomUUID().toString());
        devices.put("orgCode", "4");
        devices.put("accessChannelCount", "1");

        list.add(devices);
    }
    Map<String, Object> putParams = new HashMap<>(1);
    putParams.put("devices", list);
    // 3. Push the data of devices to DSS platform
    sendPut(buildHttpUrl("device/sync/batch"), putParams);
}

```

Example of synchronizing person groups (Java version):

```

/**
 * Synchronize person groups in batches
 * @throws Exception
 */
public static void syncPersonGroups() throws Exception {
    // 1. Obtain the third-party person groups
    BlockingQueue<Object> eventQueue = new LinkedBlockingQueue<Object>(5000);
    List<Object> handleEventList = new ArrayList<Object>();
    eventQueue.drainTo(handleEventList, 10);

    // 2. Convert the third-party person groups to DSS platform format
    List<Map<String, Object>> list = new ArrayList<>();
    for (int i=0; i<handleEventList.size(); i++) {
        Map<String, Object> personGroups = new HashMap<>(7);
        personGroups.put("orgName", "department");
        personGroups.put("parentOrgCode", "");
        personGroups.put("orgCode", "1");
        personGroups.put("remark", "remark");

        list.add(personGroups);
    }
    Map<String, Object> putParams = new HashMap<>(1);
    putParams.put("personGroups", list);
    // 3. Push the data of person groups to DSS platform
    sendPut(buildHttpUrl("person-group/sync/batch"), putParams);
}

```

Example of synchronizing people (Java version):

```

/**

```

```

* Synchronize people in batches
* @throws Exception
*/
public static void syncPersons() throws Exception {
    // 1. Obtain the third-party people
    BlockingQueue<Object> eventQueue = new LinkedBlockingQueue<Object>(5000);
    List<Object> handleList = new ArrayList<Object>();
    eventQueue.drainTo(handleList, 10);

    // 2. Convert the third-party people to DSS platform format
    List<Map<String, Object>> list = new ArrayList<>();
    for (int i=0; i<handleList.size(); i++) {
        Map<String, Object> persons = new HashMap<>(7);
        persons.put("personId", "personId");
        persons.put("lastName", "");
        persons.put("firstName", "firstName");
        persons.put("gender", "o");
        persons.put("orgCode", "1");
        persons.put("email", "email@123.com");
        persons.put("tel", "1234567890");
        persons.put("remark", "remark");
        persons.put("source", "o");
        persons.put("facePictures", "base64");
        persons.put("nickName", "nickName");
        persons.put("address", "address");
        persons.put("idType", "6");
        persons.put("idNo", "");
        persons.put("nationalityId", "9999");
        persons.put("birthday", "2000-01-01");
        persons.put("companyName", "companyName");
        persons.put("department", "department");
        persons.put("position", "position");

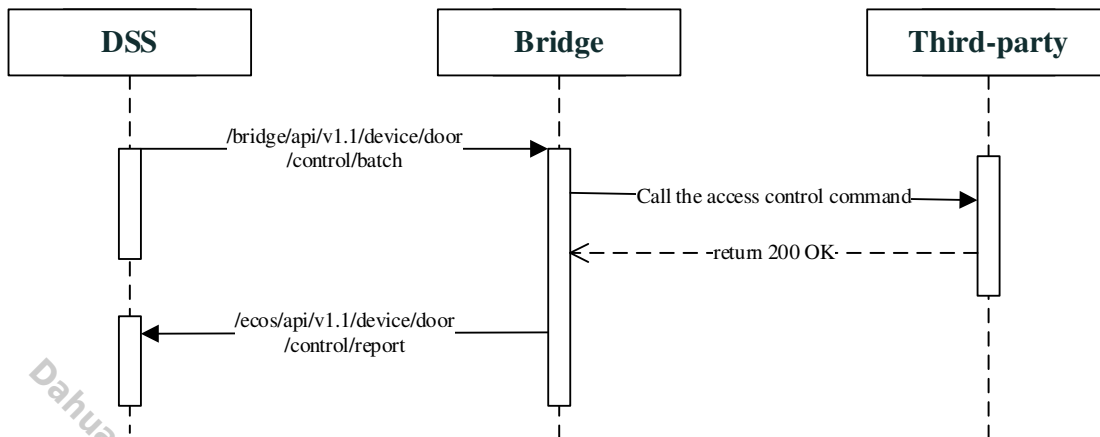
        list.add(persons);
    }
    Map<String, Object> putParams = new HashMap<>(1);
    putParams.put("persons", list);
    // 3. Push the person data to DSS platform
    sendPut(buildHttpUrl("/person/sync/batch"), putParams);
}

```

3.2.4 Access Control Operation

3.2.4.1 Workflow

DSS platform can send control commands to third-party platforms.



The specific process is as follows:

1. DSS platform sends operation commands to the Bridge. The Bridge needs to provide the `/bridge/api/v1.1/device/door/control/batch` interface to enable opening and closing instructions for third-party access control devices. DSS platform will call this interface by default.
2. Bridge sends a command to the third-party platform. If the command is not received within a specific time, a retransmission is required. If the command is successfully received, the process is completed. However, if the number of retransmissions exceeds the limit or no successful command return is received after the timeout, the command to open or close the door fails. The Bridge will log the operation for troubleshooting.
3. The operation result is synchronized with DSS platform.

3.2.4.2 Related APIs

Request path overview:

Function	URI	Request Type
Send access control device control commands (Bridge needs to provide)	<code>/bridge/api/v1.1/device/door/control/batch</code>	HTTP POST
Push the access control command interface result (return results for door opening and closing operations)	<code>/ecos/api/v1.1/device/door/control/report</code>	HTTP PUT

Note: See chapter 4 for API details.

3.2.4.3 Example of Calling API

Push the access control command interface result (return results for door opening and closing operations):

```

/**
 *
 * @throws Exception
 */
public static void reportDeviceControl() throws Exception {
    // 1. Obtain the third-party access control command result
    BlockingQueue<Object> queue = new LinkedBlockingQueue<Object>(5000);
  
```

```

List<Object> handleList = new ArrayList<Object>();
queue.drainTo(handleList, 1);

// 2. Convert the third-party access control command result to DSS platform format
for (int i=0; i<handleList.size(); i++) {
    Map<String, Object> putParams = new HashMap<>(7);
    putParams.put("channelIds", "1000001$7$0$0");
    putParams.put("operateType", "1");
    putParams.put("operateResult", "1");
    putParams.put("reportTime", "1697790234");
    putParams.put("failedReason", "");
    putParams.put("session", "1697790234302");

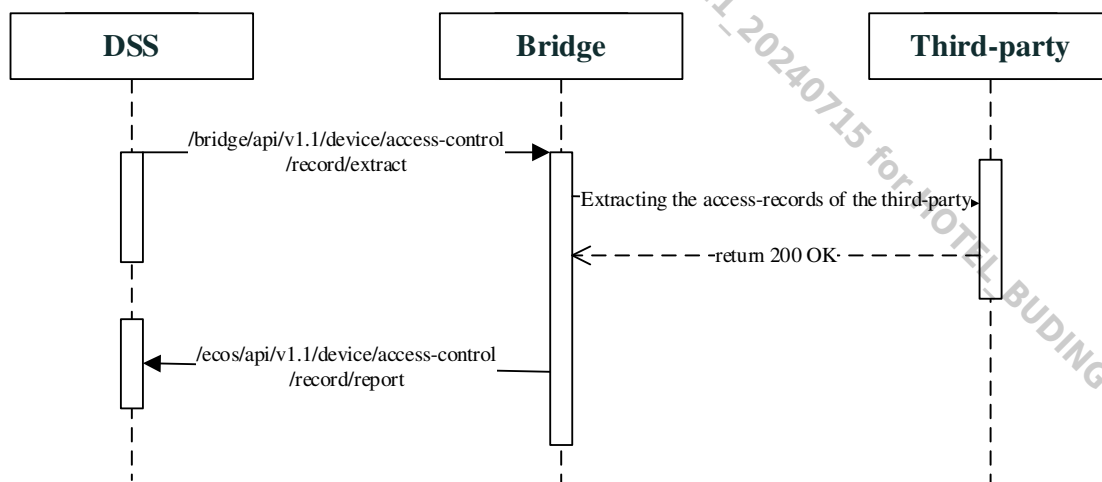
    // 3. Push the access control command result to DSS platform
    sendPut(buildHttpUrl("device/door/control/report"), putParams);
}
}

```

3.2.5 Access Control Records

3.2.5.1 Workflow

You can extract the third-party access control records from DSS platform. By default, the Bridge should provide an interface called `/bridge/api/v1.1/device/access-control/record/extract` to extract the third-party access control records. DSS platform calls this interface to synchronize the third-party access control records to the platform.



3.2.5.2 Related APIs

Request Path Overview:

Function	URI	Request Type
Extract access control device door opening records or alarm records (Bridge needs to provide)	/bridge/api/v1.1/device/access-control/record/extract	HTTP POST
Report access control device door opening records or alarm records	/ecos/api/v1.1/device/access-control/record/report	HTTP POST

Note: See chapter 4 for API details.

3.2.5.3 Example of Calling API

Report access control device door opening records or alarm records

```
/**
 * Report access control device door opening records or alarm records
 * @throws Exception
 */
public static void reportDeviceRecord() throws Exception {
    // 1. Obtain the third-party access control records
    BlockingQueue<Object> queue = new LinkedBlockingQueue<Object>(5000);
    List<Object> handleList = new ArrayList<Object>();
    queue.drainTo(handleList, 10);

    // 2. Convert the result of the third-party access control records to DSS platform format
    List<Map<String, Object>> list = new ArrayList<>();
    for (int i=0; i<handleList.size(); i++) {
        Map<String, Object> records = new HashMap<>(7);
        records.put("date", "1697696833");
        records.put("deviceCode", "7180606");
        records.put("channelId", "channelId");
        records.put("eventTypeCode", "47");
        records.put("eventName", "");
        records.put("personId", "0");
        records.put("cardNum", "");

        list.add(records);
    }
    Map<String, Object> postParams = new HashMap<>(1);
    postParams.put("taskId", "355");
    postParams.put("extractType", "1");
    postParams.put("completed", "1");
    postParams.put("extractResult", "1");
    postParams.put("extractIndex", "1");
    postParams.put("errorMessage", "");
    postParams.put("records", list);
}
```

```
// 3. Push the access control records to DSS platform
sendPost(buildHttpUrl("device/access-control/record/report"), postParams);
}
```

3.2.6 Device Status

3.2.6.1 Workflow

The device status includes the device door opening and closing status and the device offline status.

3.2.6.2 Related APIs

Request Path Overview:

Function	URI	Request Type
Push the device offline status	/ecos/api/v1.1/device/status/update/batch	HTTP POST
Push the door opening and closing results of access control device (push its door status for third-party door opening and closing events)	/ecos/api/v1.1/device/door/event/report	HTTP PUT

Note: See chapter 4 for API details.

3.2.6.3 Example of Calling API

Push the device offline status

```
/**
 * Push the device offline status
 * @throws Exception
 */
public static void pushDeviceStatus() throws Exception {
    // 1. Obtain the offline status of the third-party device
    List<String> onLineDevices = new ArrayList<>();
    List<String> offLineDevices = new ArrayList<>();

    // 2. Convert the offline result of the third-party device to DSS platform format
    List<Map<String, Object>> list = new ArrayList<>();
    Map<String, Object> onLineDeviceStatus = new HashMap<>(1);
    onLineDeviceStatus.put("deviceCodes", onLineDevices);
    onLineDeviceStatus.put("status", "1");
    onLineDeviceStatus.put("offlineReason", "99");
    list.add(onLineDeviceStatus);

    Map<String, Object> offDeviceStatus = new HashMap<>(1);
    offDeviceStatus.put("deviceCodes", offLineDevices);
    offDeviceStatus.put("status", "1");
    offDeviceStatus.put("offlineReason", "99");
}
```



```
list.add(offDeviceStatus);

Map<String, Object> postParams = new HashMap<>();
postParams.put("deviceStatus", list);
// 3. Push the device offline status to DSS platform
sendPost(buildHttpRequest("device/status/update/batch"), postParams);
}
```

Push the door opening and closing results of access control device (push its door status for third-party door opening and closing events)

```
/**
 * Push the door opening and closing results of access control device (push its switch door status for third-party door opening and
 closing events)
 * @throws Exception
 */
public static void reportDoorEvent() throws Exception {
    // 1. Obtain the door opening and closing events of the third-party device

    // 2. Analyze the door opening and closing events and report them to DSS platform
    Map<String, Object> putParams = new HashMap<>();
    putParams.put("deviceCodes", new String[]{"111111"});
    putParams.put("operateType", "1");
    putParams.put("reportTime", "1697790234");

    // 3. Push the door opening and closing status of the device to DSS platform
    sendPut(buildHttpRequest("device/door/event/report"), putParams);
}
```

3.3 Configuration Process

To complete the whole docking process and validate the accuracy of the business process, it is necessary to add a Project to the DSS client and configure the event type.

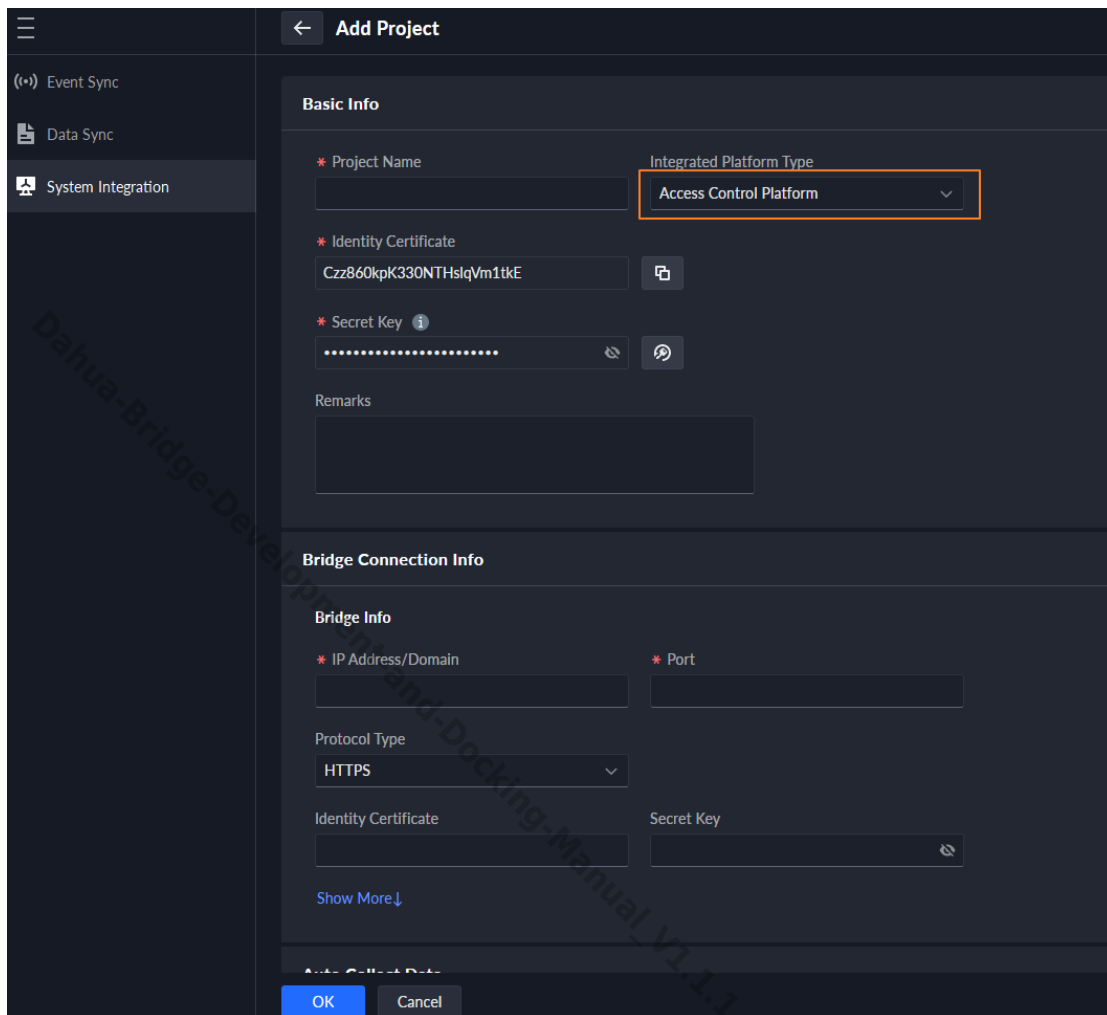
3.3.1 Adding Project

Select  > **Synthesis > System Integration > Add Now.**

Enter the project name and set **Access Control Platform** to **Integrated Platform Type**. To enhance security, the platform automatically generates a pair of identity credentials and accompanying keys. These keys can be reset if needed. To view the plaintext and reset the keys, you need to enter the login password. Additionally, both fields can be copied for convenient use in future maintenance of the Bridge configuration file. The subsequent authentication of the Bridge to the platform will require these specific credentials and keys to be carried.

Enter the IP address and port of the Bridge. The Bridge can also maintain a pair of identity credentials and keys for authentication when the platform connects to the Bridge. Leave its value as empty as the current Bridge does not support the function.

The default value is required for other fields. Click **OK** to save the project information.



3.3.2 Synchronizing Event Types

Once the Project is added, we need to add the third-party event types to be reported on DSS platform to the project.

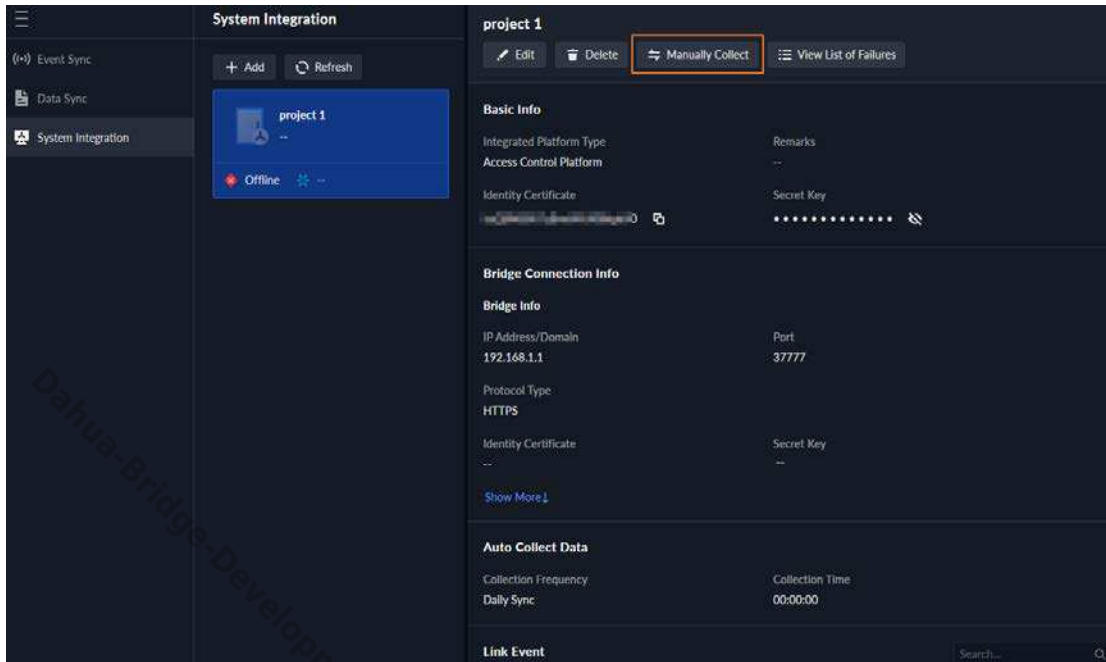
DSS supports adding event types one by one or in batches. Once added, you can view the event types on the Project page.

3.3.3 Deploying Bridge

Deploy the Bridge on the designated server based on your own logic, ensuring that it can be accessed by both DSS platform and third-party platforms.

3.3.4 Synchronizing Data

After adding the project to DSS platform, ensure that the Bridge is connected to both DSS platform and the third-party platform, and that it can be accessed without any issues. Click **Manually Collect** to synchronize the device organization, devices, person groups, and people on the platform.



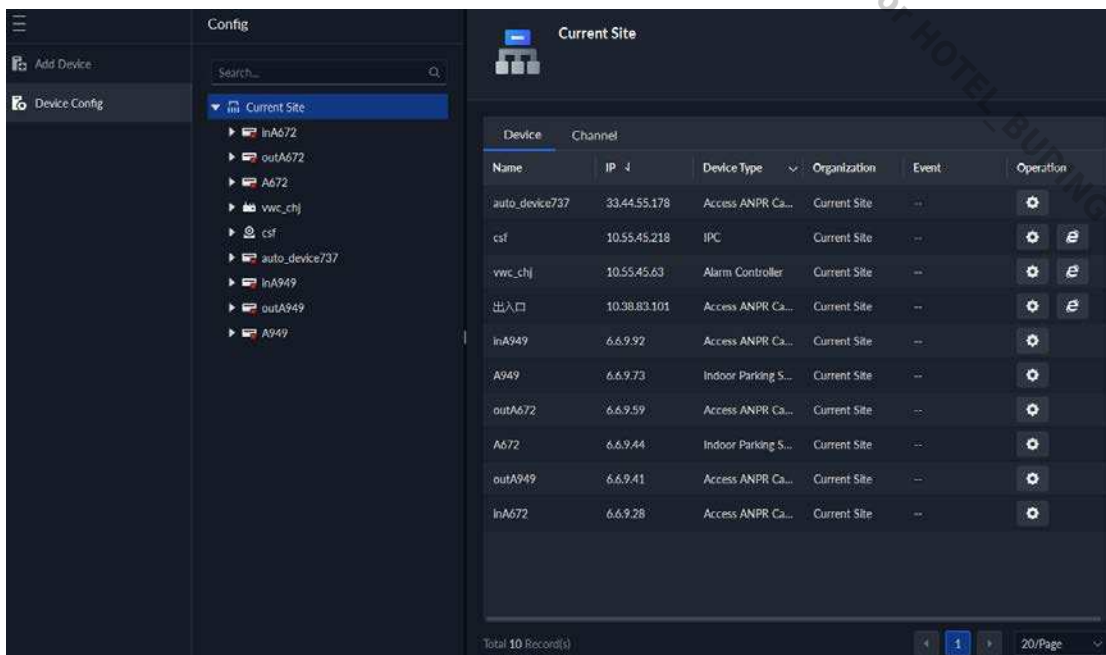
Once the synchronization is complete, refresh the page to view the synchronization status. If the synchronization is successful, it means that the basic data has been successfully synchronized.

Please note that there is also an option of **Auto Collect Data**, where device and person information is automatically synchronized at 00:00 every day. You can modify this information in the Project as needed.

Make sure to synchronize the device organization first and then the devices. Once the third-party devices are successfully synchronized, you can access various application interfaces such as the **Device, Monitoring Center**, and **Access Control**. The third-party organization will be displayed with unique icons.

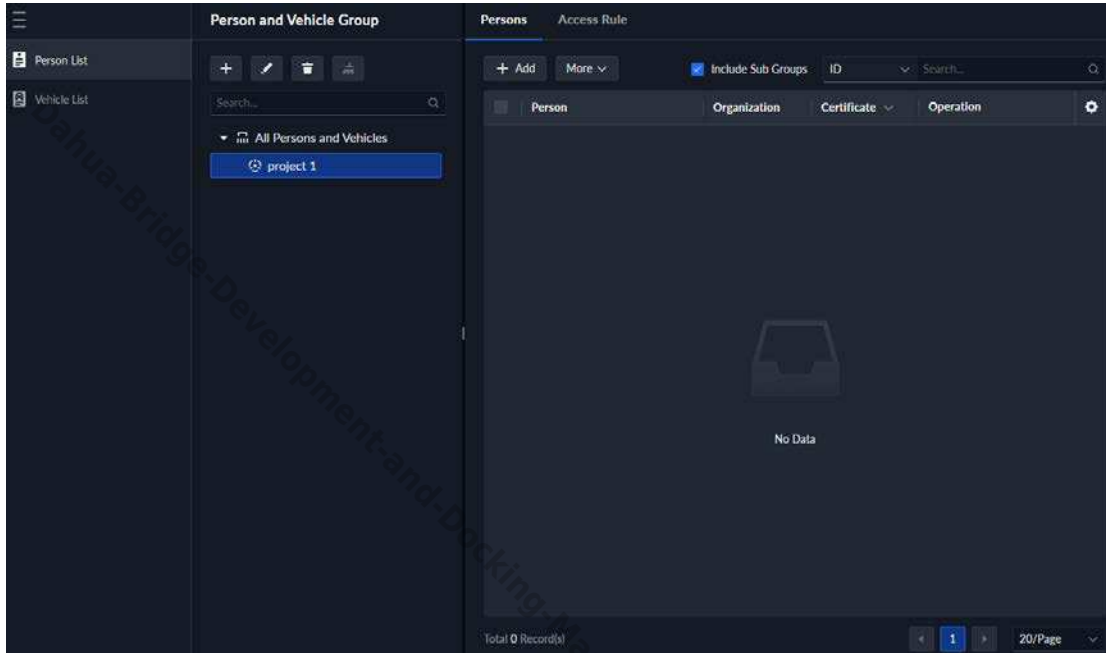
Other notes:

1. The **Device** module does not display the third-party devices.
2. Third-party access control devices automatically generate access points, but access rules cannot be configured.
3. You can drag the access point to the map. When an alarm is reported, the map will be triggered to flicker.



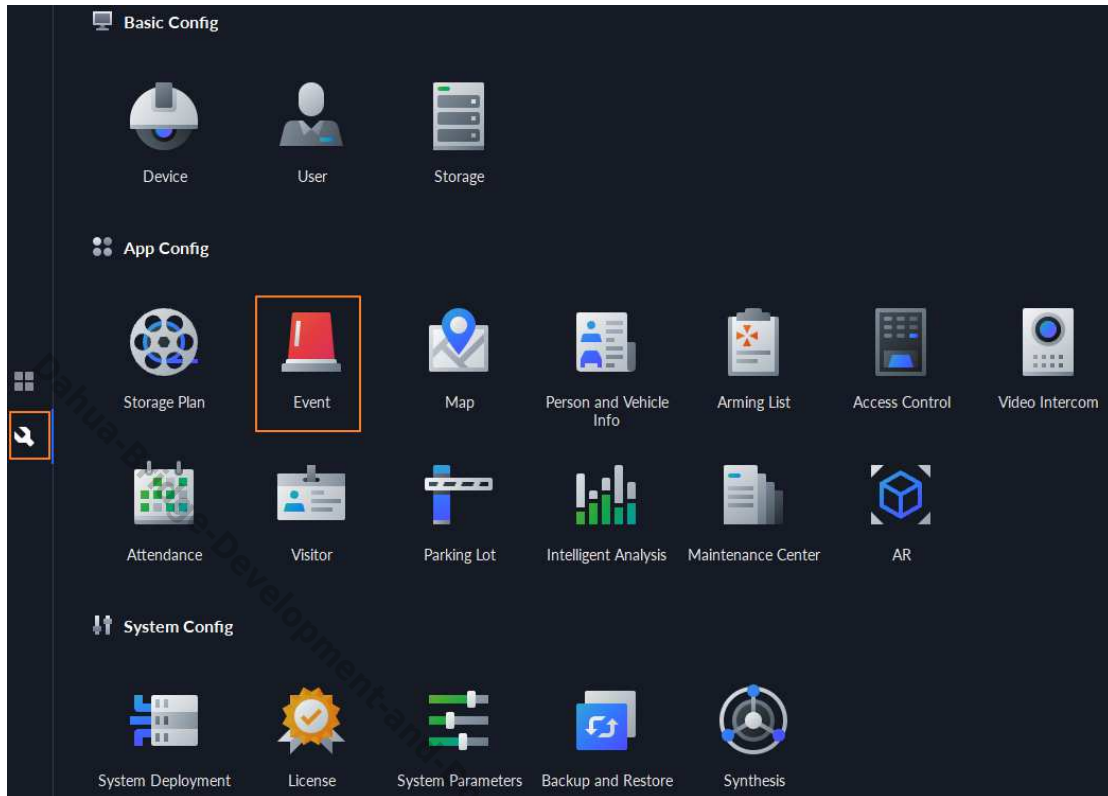
Make sure to synchronize person organizations first, and then synchronize person. After the third-party people are synchronized, you can view them in the **Persons** module. A special icon is displayed in the third-party organization.

Third-party people are unable to move or delete their person groups and edit basic information directly on the platform. However, their person information can be sent to the devices of DSS platform. If you need to change the basic person information, modify it in a third-party platform and synchronize it through Bridge.

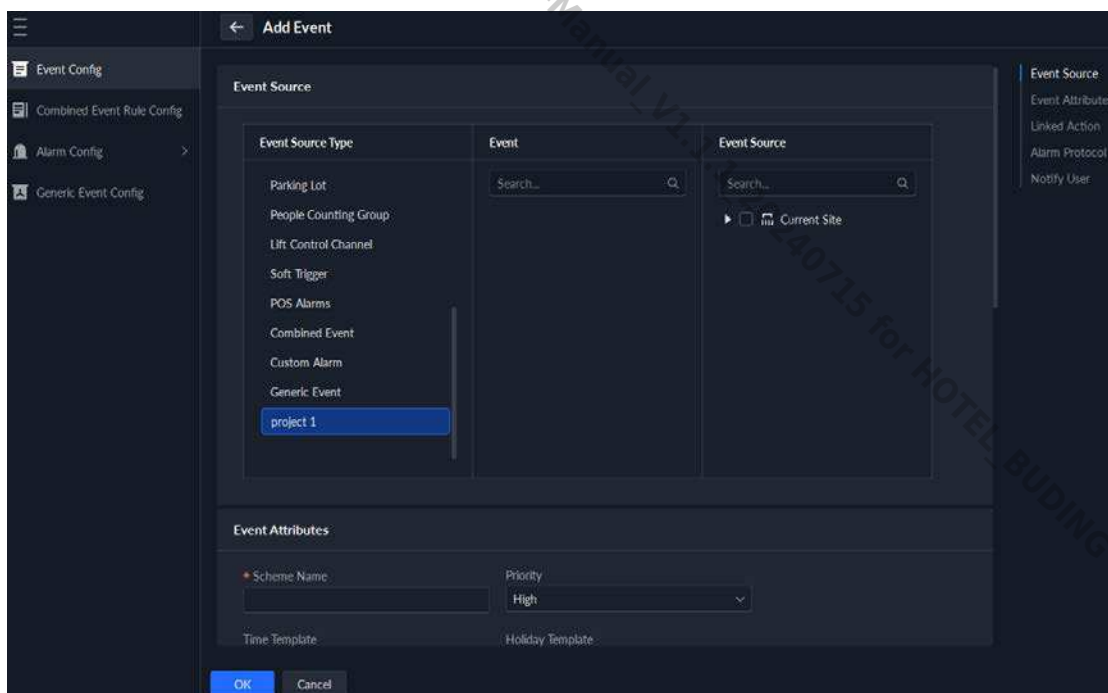


3.3.5 Configuring Event

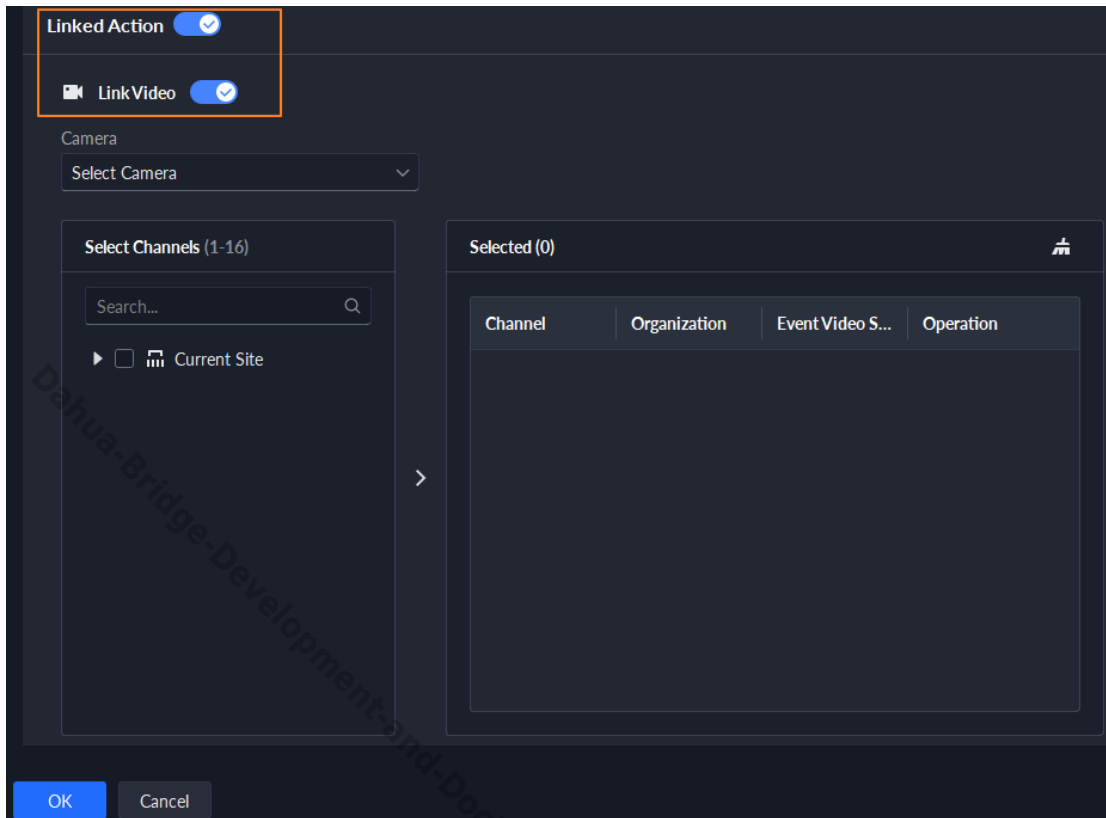
Select  > Event > Event Configuration > Add Event.



Select the event type and event source.



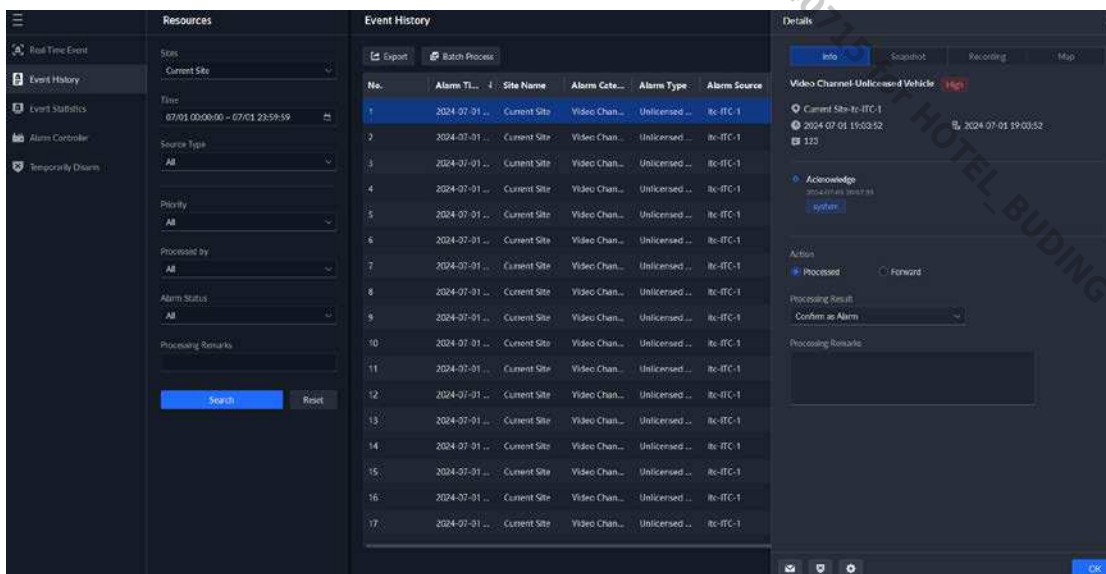
Enter the event name. Enable **Linked Action** and **Linked Video**. Select the video to be linked.



Click **OK** to add the event.

3.3.6 Reporting Events

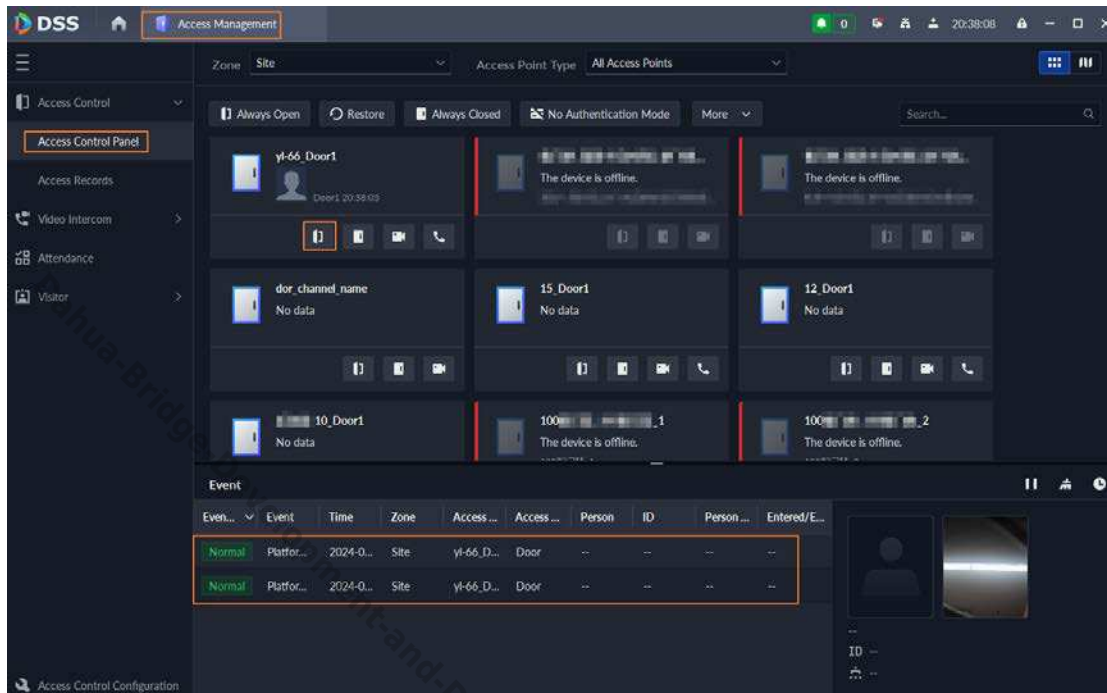
When an event arises on a third-party platform, it is pushed to DSS platform in real time. DSS platform determines whether the event is reported on the platform based on the event type and event source configured in the event synchronization, and links it with the corresponding video. You can view subsequent events in the **Event History**.



3.3.7 Access Control Operations

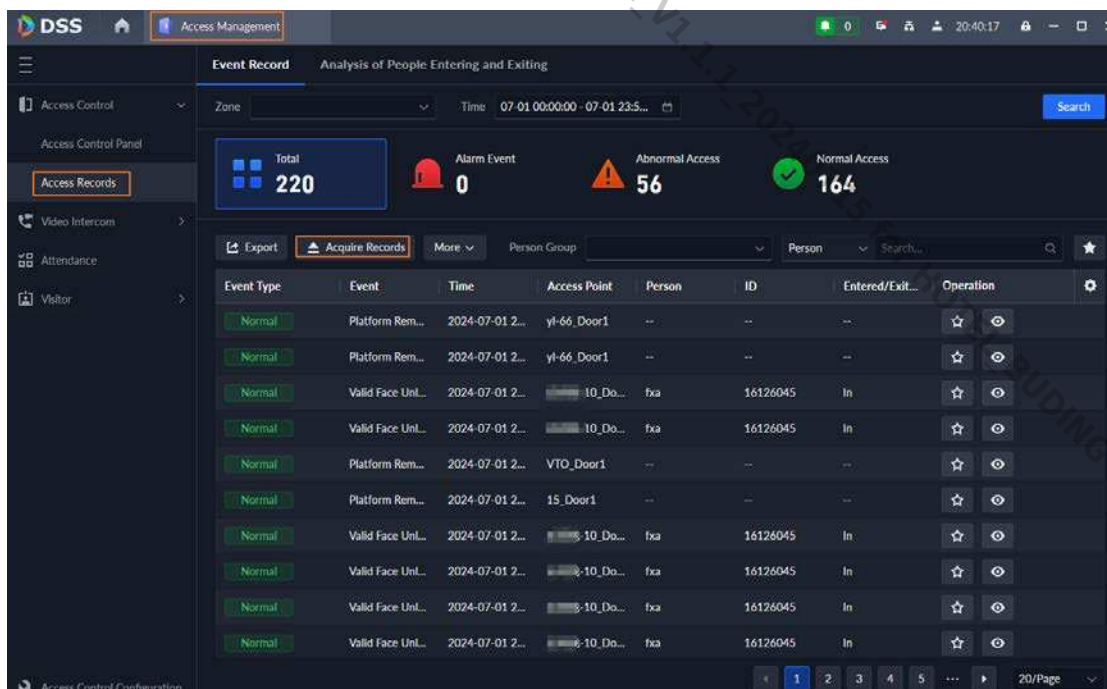
Select **Access Management > Access Control Panel**.

Click the open and close button to switch the door. The corresponding door opening and closing events will be displayed at the **Event** section.

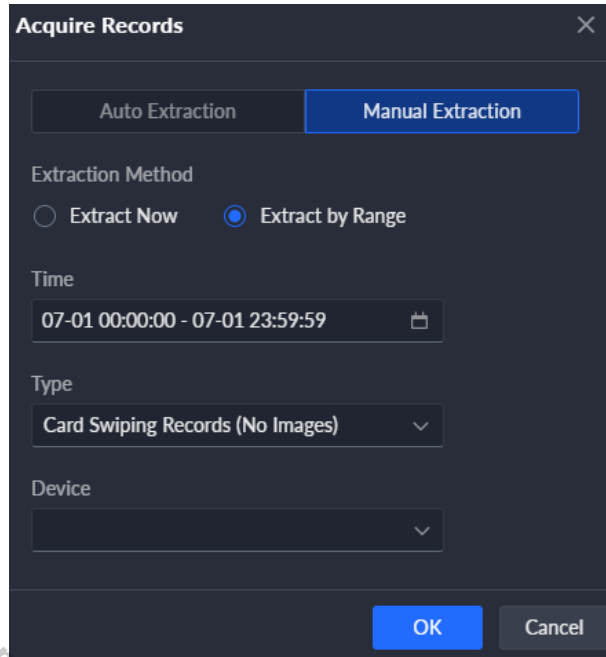


3.3.8 Access Control Records

Select **Access Management > Access Records**, and then click **Acquire Records**. Enter the password for DSS-PRO to obtain the records.



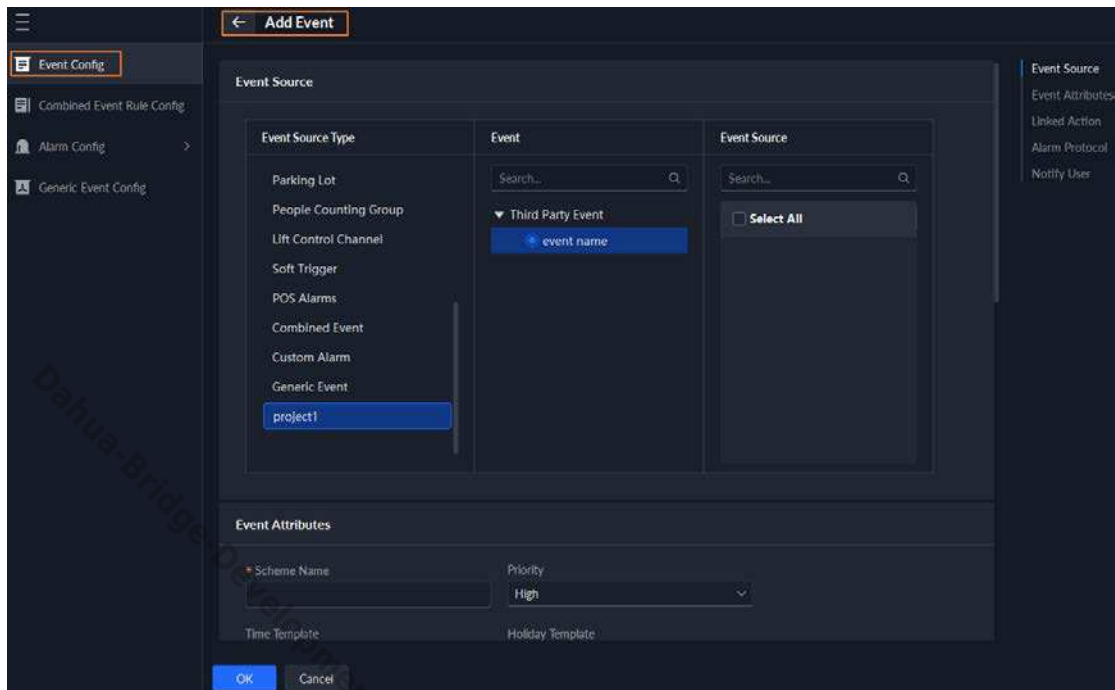
Auto Extraction and **Manual Extraction** are available. For **Manual Extraction**, you can select **Extract by Range**, and configure the time, type, and device. This allows you to extract the corresponding access control or alarm records.



After the records are successfully acquired, a prompt message will be displayed at the upper-right page.

3.4 Test Process

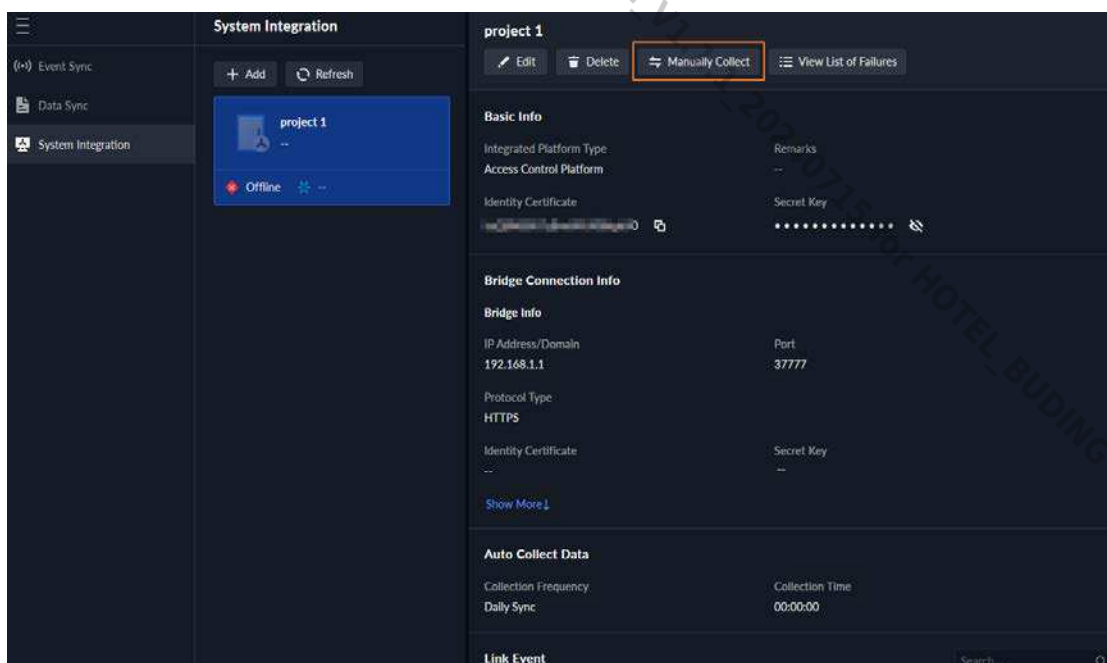
- 1) Add a Bridge on the DSS client and configure system integration information.
- 2) Deploy the Bridge and reasonably configure the connection information between DSS platform and third-party systems, such as IP, port, access key, secret key, username, and password. Ensure that both systems can be successfully logged in.
- 3) Select  > **App Config** > **Event** on the DSS client and select the configured Bridge to add an event scheme. You can configure multiple event schemes based on the number of triggered events, as shown in the following figure. When configuring **Linked Action** according to business needs, you must select the **Notify User** check box, which refers to the user who finally receives the event.



4) Trigger events in third-party systems.

5) Select **Applications > Event Center > Real-time Events** on the DSS client to check whether the third-party events are received. If they are received, the test is completed, otherwise the troubleshooting shall be carried out according to the FAQ.

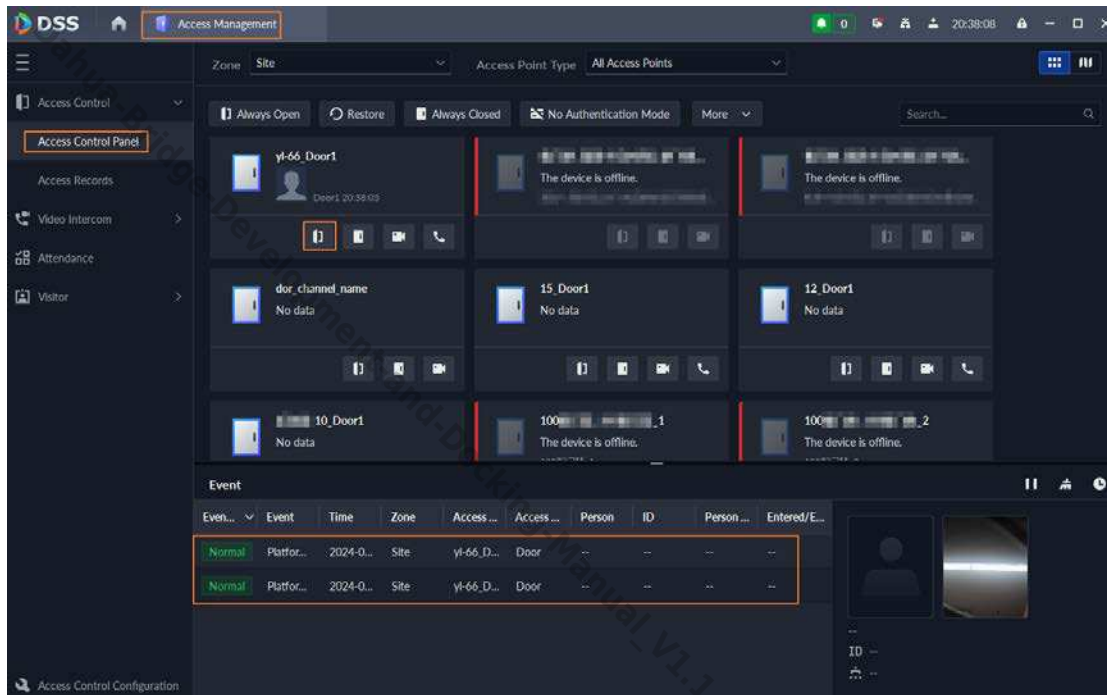
6) Select  > **Synthesis > System Integration**, and then click **Manually Collect**.



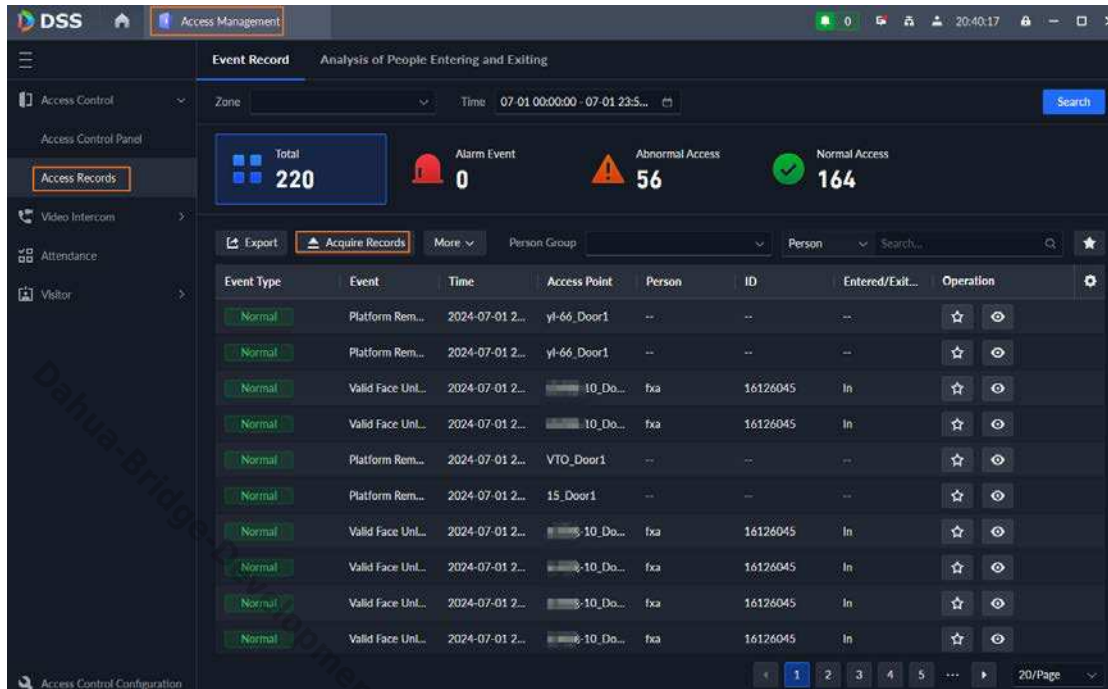
7) Select  > **Device > Device Config** on the DSS client to check whether the third-party devices are synchronized. Select  > **Person and Vehicle Info > Person List** to check whether the third-party people

are synchronized. If the synchronization is not correct, select  > **Synthesis** > **System Integration**, and then click **View List of Failures** in the corresponding Project to confirm the specific cause of the failure.

8) Select  > **Access Management** > **Access Control** > **Access Control Panel** on the DSS client, and then click the open and close button. By doing so, you can check whether DSS platform has reported the door opening and closing events, and whether the access control device linked to the third-party platform is normally opened and closed. If the door is opened and closed normally, it indicates that the function testing of access control is complete.



9) Select  > **Access Management** > **Access Control** > **Access Records** on the DSS client, and then click **Acquire Records**. Enter the password. After a certain period of time, verify whether the third-party access control records are accurately synchronized with DSS platform. If this is the case, it confirms that the function testing of access control record extraction is complete.



4. Description of Related API Interfaces

4.1 Authorization Authentication

1) Authorization authentication

Port Name	/ecos/api/v1.1/account/authorize			
Port Description	Uses AK/SK mode for authentication and verification of user permissions, and issues tokens if the verification is passed.			
Interface method	POST			
Input Parameter	Field Code	Data Type	Null or not	Field Description
	accessKey	String	Y	AK, the unique identifier of access
	signature	String	Y	Signature, generated based on the HMAC-SHA256 (timestamp, secretKey) algorithm
	timestamp	Long	Y	Timestamp
Input example	<pre>{ "accessKey": "1", "signature": "a34ed686d54b45521a0e12b7852354f38559b112fe1ff878fb86dc92feec31d4", "timestamp": 1374908054 }</pre>			
Output Parameter	Field Code	Data Type	Field Description	
	token	String	Token	
	duration	Int	Survival time (seconds)	
	version	String	Version	

Output Example	<pre>{ "code": 1000, "desc": "Success", "data": { "token": "a15ee202efab4a54af01c5coa4ed9bac", "duration": 1800 } }</pre>
----------------	---

2) Update Token

Port Name	/ecos/api/v1.1/account/token/update			
Port Description	Issues a new token. And the original token will automatically expire after being retained for 30 minutes. We recommend calling this interface every 25 minutes.			
Interface method	POST			
Input Parameter	Field code	Data Type	Null or not	Field description
	signature	String	Y	Signature, generated based on the HMAC-SHA256(timestamp, secretKey) algorithm
	timestamp	Long	Y	Timestamp
	token	String	Y	Token
Input Example	{ "signature": "a34ed686d54b45521a0e12b7852354f38559b112fe1ff878fb86dc92feec31d4", "timestamp": 1374908054, "token": "a15ee202efab4a54af01c5coa4ed9bac" }			
Output Parameter	Field code	Data Type	Field description	
	token	String	Token	
	duration	Int	Valid time of token (seconds)	
Output Example	{ "code": 1000, "desc": "Success", "data": { "token": "a15ee202efab4a54af01c5coa4ed9bac", "duration": 1800 } }			

3) Keep the heartbeat

Port Name	/ecos/api/v1.1/account/heartbeat			
Port Description	Heartbeat keeps alive. We recommend calling it every 30 seconds.			
Interface method	POST			
	Field Code	Data Type	Null or not	Field Description

Input Parameter	No			
Input example				
Output Parameter	Field code	Data Type	Field Description	
	No			
Output Example	{ "code": 1000, "desc": "Success" }			

4) Obtain the configuration information of third-party

Port Name	/ecos/api/v1.1/bridge/{id}/config			
Port Description	After registering with the bridge platform, obtain login information for connecting to third parties from the platform.			
Interface method	GET			
Input Parameter	Field Code	Data Type	Null or not	Field Description
	No			
Input example				
Output Parameter	Field Code	Data Type		Field Description
	id	String		Bridge ID
	bridgeName	String		Bridge name
	bridgeType	String		The types of bridge are as follows: 1=event synchronous connector, 2=system integration connector
	platformType	String		Sub-platform type
	thirdPartyIp	String		Third-party IP
	thirdPartyPort	String		Port number of third-party
	thirdPartyAuthKey	String		Login account for third-party systems, such as AccessKey and username
	thirdPartyAuthValue	String		Login password of third-party system, such as SecretKey and password.
Output example	{ "code": 1000, "desc": "Success", "data": { "id": "1", "bridgeName": "event-sync-demo",			

	<pre> "bridgeType": "1", "platformType": null, "thirdPartyIp": "10.56.11.163", "thirdPartyPort": "443", "thirdPartyAuthKey": "system", "thirdPartyAuthValue": "16uaOeRopnrecsPGuU1AsA==" } </pre>
--	---

4.2 Event Push

1) Push events

Port Name	/ecos/api/v1.1/bridge/event/push			
Port Description	Push event messages.			
Interface Method	POST			
Input Parameter	Field Code	Data Type	Null or not	Field Description
	eventId	String	Y	Event code, unique in the third-party system. If the third-party system does not have the code, it can randomly generate a UUID
	eventTime	String	Y	Event Timestamp, in seconds
	eventSourceCode	String	Y	Event source code, corresponding to the configured event source code
	eventSourceName	String	N	Event source name. If it is inconsistent with the configuration, the configuration shall prevail.
	eventSourceStatus	String	N	Event source status
	eventTypeCode	String	Y	Event type code, corresponding to the configured event type code
	eventName	String	N	Event type name. If it is inconsistent with the configuration, the configuration shall prevail.
	pictureCount	String	N	Number of event images
	videoCount	String	N	Number of event videos
	userId	String	N	Person ID
	cardNo	String	N	Card number
	readerId	String	N	Card reader ID
	remark	String	N	Remark
Input Example	<pre> { "eventId": "{C6FF341F-2BoD-425B-B788-39E9E2606AE9}", "eventTime": "1615248000", "eventSourceCode": "1000005\$1\$0\$2", "eventSourceName": "Entrance Door", "eventSourceStatus": "2", </pre>			

	<pre> "eventTypeCode": "51", "eventName": "Valid Swipe", "pictureCount": "2", "videoCount": "2", "remark": "This is an event" </pre>		
Output Parameter	Field Code	Data Type	Field Description
	No		
Output Example	<pre> { "code": 1000, "desc": "Success" } </pre>		

2) Push files in batches

Port Name	/ecos/api/v1.1/bridge/event/file/push/batch			
Port Description	Push the files (images, videos) carried by the event.			
Interface Method	POST			
Input Parameter	Field Code	Data Type	Null or Not	Field Description
	associateId	String	Y	Event encoding, unique within the third-party system. It needs to be consistent with the eventId of the "Push events" interface
	files	Array	Y	File list
	-- fileUrl	String	Y	The outside address of file, only http(s) is supported, and the request URL does not require authentication
	-- fileType	String	Y	File type: 0 = Unknown, 1 = Video, 2 = Audio, 3 = Picture
	-- recordTime	String	N	Video file recording start time, timestamp, in seconds
	-- fileTimeLength	String	N	The time of the video, in seconds
Input Example	-- fileLength	String	N	File size, in bytes
	<pre> { "associateId": "C6FF341F2BoD425BB78839EgE26o6A", "files": [{ "fileUrl": "https://file.jpg", "fileType": "3", "recordTime": "", "fileTimeLength": "" }] } </pre>			

	<pre> "fileLength": "234"] } </pre>		
Output Parameter	Field Code	Data Type	Field Description
	Not		
Output Example	<pre> { "code": 1000, "desc": "Success" } </pre>		

3) Push events in batches (applicable to high-concurrency scenarios)

Port Name	/ecos/api/v1.1/bridge/event/push/batch			
Port Description	Push event messages in batches.			
Interface Method	POST			
Input Parameter	Field Code	Data Type	Null or not	Field Description
	events	Array	Y	Event list
	-- eventId	String	Y	Event code, unique in the third-party system. If the third-party system does not have the code, it can randomly generate a UUID
	-- eventTime	String	Y	Event Timestamp, in seconds
	-- eventSourceCode	String	Y	Event source code, corresponding to the configured event source code
	-- eventSourceName	String	N	Event source name. If it is inconsistent with the configuration, the configuration shall prevail.
	--eventSourceStatus	String	N	Event source status
	-- eventTypeCode	String	Y	Event type code, corresponding to the configured event type code
	-- eventTypeName	String	N	Event type name. If it is inconsistent with the configuration, the configuration shall prevail.
	-- pictureCount	String	N	Number of event images
	-- videoCount	String	N	Number of event videos
	-- userId	String	N	Person ID
	-- cardNo	String	N	Card number
	-- readerId	String	N	Card reader ID
	-- remark	String	N	Remark
	<pre> { "events": [{ "eventId": "{C6FF341F-2BoD-425B-B788-39E9E2606AE9}", "eventTime": 1615248000, </pre>			

Input Example	<pre> "eventSourceCode": "1000005\$1\$0\$2", "eventSourceStatus": "2", "eventSourceName": "Entrance Door", "eventTypeCode": "51", "eventName": "Valid Swipe", "pictureCount": "2", "videoCount": "2", "remark": "This is an event" }] } </pre>		
Output Parameter	Field Code	Data Type	Field Description
	No		
Output Example	<pre> { "code": 1000, "desc": "Success" } </pre>		

4.3 Synchronizing Data

1) Synchronize device organizations

Port Name	/ecos/api/v1.1/organization/sync/batch			
Port Description	Synchronize device organizations.			
Interface method	POST			
Input Parameter	Field code	Data Type	Null or not	Field Description
	organizations	Array	N	Third-party device organization list
	-- orgName	String	Y	Third-party device organization name
	-- parentOrgCode	String	N	Third-party parent device organization code
	-- orgCode	String	Y	Third-party device organization code
	-- remark	String	N	Remark
Input Example	<pre> { "organizations": [{ "parentOrgCode": "", "orgName": "DoorGroup333", "remark": "", "orgCode": "4" }] } </pre>			

Output Parameter	Field Code	Data Type	Field Description
	No		
Output Example	<pre>{ "code": 1000, "desc": "Success" }</pre>		

2) Synchronize Devices

Port Name	/ecos/api/v1.1/device/sync/batch			
Port Description	Synchronize devices.			
Interface Method	PUT			
Input Parameter	Field Code	Data Type	Null or not	Field Description
	devices	Array	N	Third-party device list
	-- addType	String	Y	Add methods for third-party devices: 1 = IP address, 4 = active registration
	-- deviceIp	String	N	Device IP
	-- deviceName	String	Y	Third-party device name
	-- devicePort	String	N	Third-party device port
	-- deviceCategory	String	Y	Third-party device category
	-- deviceType	String	Y	Third-party device subcategory
	-- associateId	String	Y	Third-party device ID
	-- registerId	String	N	The registration ID of the third-party device. Required when P2P and active registration methods are accessed.
	-- orgCode	String	Y	Third-party device organization code
	-- accessChannelCount	String	N	Number of access control channels
Input Example	<pre>{ "devices": [{ "addType": "4", "deviceIp": "-", "deviceCategory": "8", "deviceName": "0902-2", "orgCode": "", "associateId": "222224", "deviceType": "99", "accessChannelCount": "1", "registerId": "a852848994", "devicePort": "3777" }] }</pre>			

	{		
Output Parameter	Field Code	Data Type	Field Description
	No		
Output Example	<pre>{ "code": 1000, "desc": "Success" }</pre>		

3) Synchronize person groups

Port Name	/ecos/api/v1.1/person-group/sync/batch			
Port Description	Synchronize person groups.			
Interface Method	PUT			
Input Parameter	Field Code	Data Type	Null or not	Field Description
	personGroups	Array	N	Third-party person group list
	-- orgName	String	Y	Third-party person group name
	-- parentOrgCode	String	N	Third-party parent person group code
	-- orgCode	String	Y	Third-party person group code
	-- remark	String	N	Remark
Input Example	{ "personGroups": [{ "parentOrgCode": "", "orgName": "department134", "remark": "", "orgCode": "1" }] }			
Output Parameter	Field Code	Data Type	Field Description	
	No			
Output Example	{ "code": 1000, "desc": "Success" }			

4) Synchronize person

Port Name	/ecos/api/v1.1/person/sync/batch
Port Description	Synchronize person

Interface Method	PUT			
Input Parameter	Field Code	Data Type	Null or not	Field Description
	persons	Array	N	Third-party person list
	-- personId	String	Y	Third-party person ID
	-- lastName	String	N	Last name
	-- firstName	String	Y	First name
	-- gender	String	Y	Gender: 0= 1= Male, 2= Female, 0 by default
	-- orgCode	String	N	Person group code
	-- email	String	N	Email
	-- tel	String	N	Phone number
	-- remark	String	N	Remark
	-- source	String	N	Person source: 0 = Person management list, 1 = Access control group, 2 = Face comparison group, 3 = Entrance and exit vehicle group, 4 = Visitor, 5 = Acquired from devices, 0 by default.
	-- facePictures	Array	N	Face image data Base64
	-- nickName	String	N	Nickname
	-- address	String	N	Address
	-- idType	String	N	The certificate type: 0 = ID card, 1 = Certificate of officer, 2 = Student card, 3 = Driving license, 4 = Passport, 5 = Taxpayer identification number, 6 = Others, 6 by default.
	-- idNo	String	N	ID number
	-- nationalityId	String	N	Country/region number, default: 9999 = unknown
	-- birthday	String	N	Birthday, YYYY-MM-DD
	-- companyName	String	N	Company name
	-- department	String	N	Department
	-- position	String	N	Position
Input Example	<pre>{ "persons": [{ "personId": "1", "lastName": "", "firstName": "User111", "gender": "0", "orgCode": "", "email": "", "tel": "1234567899", "remark": "", "source": "0",</pre>			

	<pre> "facePictures": null, "nickName": "", "address": "", "idType": "6", "idNo": "1339", "nationalityId": "9999", "birthday": "", "companyName": "", "department": "", "position": "" } } </pre>		
Output Parameter	Field Code	Data Type	Field Description
	No		
Output Example	<pre> { "code": 1000, "desc": "Success" } </pre>		

The interfaces provided by Bridge are as follows:

5) Perform third-party platform data synchronization tasks

Port Name	/bridge/api/v1.1/task/data/sync			
Port Description	Perform third-party platform data synchronization tasks.			
Interface method	POST			
Input Parameter	Field Code	Data Type	Null or not	Field Description
	taskId	String	Y	Basic data synchronization task ID
Input Example	{ "taskId": 3 }			
Output Parameter	Field Code	Data Type	Field Description	
	No			
Output Example	{ "code": 1000, "desc": "Success" }			

4.4 Access Control Operation

It can be divided into interfaces of DSS platform itself and interfaces that Bridge needs to provide.

DSS platform interfaces are as follows:

1) Push the access control command interface result (return results for door opening and closing operations)

Port Name	/ecos/api/v1.1/device/door/control/report			
Port Description	Push the access control command interface result (return results for door opening and closing operations).			
Interface Method	PUT			
Input Parameter	Field Code	Data Type	Null or not	Field Description
	channelIds	Array	Y	Device channel code list
	operateType	String	Y	Operation type: 1 = Access control device open, 2 = Access control device close, 3 = Access control device always closed, 4 = Access control device always opened, 5 = Access control device restored
	operateResult	String	Y	Operation result: 1= Successful, 2 = Failed
	reportTime	String	Y	Operation result reporting time, in seconds
	session	String	Y	Session ID
	failedReason	String	N	Failure reason: 1 = The third-party platform does not support it, 2 = Failed to send the command
Input Example	{ "channelIds": ["1000001\$7\$0\$0"], "operateType": "1", "operateResult": "1", "reportTime": "1697790234", "failedReason": "", "session": "1697790234302" }			
Output Parameter	Field Code	Data Type	Field Description	
	No			
Output Example	{ "code": 1000, "desc": "Success" }			

The interfaces provided by Bridge are as follows:

2) Send access control device control commands in batches

Port Name	/bridge/api/v1.1/device/door/control/batch			
Port Description	Send access control device control commands in batches.			
Interface method	POST			
Input Parameter	Field Code	Data Type	Null or not	Field Description
	controlDevices	Array	Y	
	-- deviceCode	String	Y	Third-party device code
	-- channelIds	Array	N	Third-party device channel code

	-- operateType	String	Y	Operation type: 1 = Access control device open, 2 = Access control device close, 3 = Access control device always closed, 4 = Access control device always opened, 5 = Access control device restored
Input Example	<pre>{ "controlDevices": [{ "session": "1697788242715", "operateType": "2", "deviceCode": "7180606", "channelIds": ["1000001\$7\$0\$0"] }] }</pre>			
Output Parameter	Field Code	Data Type	Field Description	
	No			
Output Example	<pre>{ "code": 1000, "desc": "Success" }</pre>			

4.5 Access Control Records

It can be divided into interfaces of DSS platform itself and interfaces that Bridge needs to provide.

DSS platform interfaces are as follows:

- 1) Report access control device door opening records or alarm records

Port Name	/ecos/api/v1.1/device/access-control/record/report			
Port Description	Report access control device door opening records or alarm records.			
Interface Method	POST			
Input Parameter	Field Code	Data Type	Null or not	Field Description
	taskId	String	Y	Extract task ID
	extractType	String	Y	Extracted record type: 1 = Access control records, 2 = Alarm records
	completed	String	Y	Whether the record extraction is completed: 0 = No, 1 = Yes
	extractResult	String	Y	Whether the record extraction is successful: 0 = No, 1 = Yes
	extractIndex	String	Y	Record extraction batch, starting from 1
	errorMessage	String	N	Description of extraction failure
	records	Array	Y	Access control records
	-- date	String	Y	Record time

	-- deviceCode	String	Y	Device code
	-- channelId	String	N	Channel ID
	-- eventTypeCode	String	Y	Event type code
	-- eventTypeName	String	Y	Event type name
	-- personId	String	N	Person ID
	-- cardNum	String	N	Card number
Input Example	<pre>{ "taskId": "355", "extractType": "1", "completed": "1", "extractIndex": 1, "extractResult": "1", "errorMessage": "", "records": [{ "date": "1697696833", "deviceCode": "7180606", "channelId": "", "eventTypeCode": "47", "eventName": "", "personId": "0", "cardNum": "" }] }</pre>			
Output Parameter	Field Code	Data Type	Field Description	
	No			
Output Example	<pre>{ "code": 1000, "desc": "Success" }</pre>			

The interfaces provided by Bridge are as follows:

2) Extract access control device door opening records or alarm records

Port Name	/bridge/api/v1.1/device/access-control/record/extract			
Port Description	Extract access control device door opening records or alarm records.			
Interface Method	POST			
Input Parameter	Field Code	Data Type	Null or not	Field Description
	taskId	String	Y	Extract task ID
	beginTime	String	Y	Extraction start time, in seconds
	endTime	String	Y	Extraction end time, in seconds
	deviceCodes	Array	Y	Device code

	extractType	String	N	Type of extracted records: 1 = Access control records, 2 = Alarm records. If it is empty, both door opening records and alarm records are acquired.
	eventTypes	Array	N	Third-party event type code
Input Example	{ "beginTime": "1697731200", "endTime": "1697817599", "deviceCodes": ["7180606"], "eventTypes": ["0", "1", "2"], "extractType": "1", "taskId": "247" }			
Output Parameter	Field Code	Data Type	Field Description	
	No			
Output Example	{ "code": 1000, "desc": "Success" }			

4.6 Device Status

1) Push the device offline status

Port Name	/ecos/api/v1.1/device/status/update/batch			
Port Description	Push the device offline status.			
Interface Method	PUT			
Input Parameter	Field Code	Data Type	Null or not	Field Description
	deviceStatus	Array	Y	
	--deviceCodes	Array	Y	Device code list
	--status	String	Y	The device status: 0 = Offline, 1 = Online
	--offlineReason	String	N	The offline reason
Input Example	<pre>{ "deviceStatus": [{ "deviceCodes": ["111111"], "status": "0", "offlineReason": "99" }] }</pre>			

	<pre>] } </pre>		
Output Parameter	Field Code	Data Type	Field Description
	No		
Output Example	<pre> { "code": 1000, "desc": "Success" } </pre>		

2) Push the door opening and closing results of access control device (push its door status for third-party door opening and closing events)

Port Name	/ecos/api/v1.1/device/door/event/report			
Port Description	Push the door opening and closing results of access control device (push its door status for third-party door opening and closing events)			
Interface Method	PUT			
Input Parameter	Field Code	Data Type	Null or not	Field Description
	deviceCodes	Array	Y	Third-party device code list
	operateType	String	Y	Operation type: 1 = Access control device open, 2 = Access control device close, 3 = Access control device always closed, 4 = Access control device always opened, 5 = Access control device restored
	reportTime	String	Y	Operation result reporting time, in seconds
Input Example	{ "deviceCodes": "1000001", "operateType": "1", "reportTime": "1697790234" }			
Output Parameter	Field Code	Data Type	Field Description	
	No			
Output Example	{ "code": 1000, "desc": "Success" }			

5. FAQ

Q: Why can't I find the **Synthesis** menu after logging in to the DSS client?

A: Only super administrator and administrator have the permission to configure the Bridge. Common users are not allowed to configure the Bridge, so the menu is blocked for common users. Please confirm whether the current login user is an administrator.

Q: Why is the status in the Bridge list always Disconnected?

A: When developing a Bridge, you have to call the /heartbeat interface of DSS platform continuously after the authentication is completed. Then, you can see the connection status of the Bridge on the DSS client. However, this interface is optional. If it is not called, the connection status of the Bridge will not be displayed correctly on the DSS client, but the event push will not be affected.

Q: Bridge fails to initiate the authentication to DSS platform. How to troubleshoot that?

A: The Access Key displayed on the DSS client is in plain text, while the Secret Key is encrypted. The Secret Key should be displayed in plain text first. Then, you can use the copy function on the client to get the two strings and configure them into the configuration file of the Bridge. In addition, the signature parameter of the /authorize interface is encrypted with the HMAC-SHA256 algorithm. The signature algorithm rules of Bridge should be exactly the same as those of DSS platform. You can practice the verification by using the examples provided in section 2.1.1 of the document. After the authentication is successfully completed, the /token/update interface must be called regularly. The validity period of the authentication token is only 30 minutes.

Q: The third-party system triggers the event, but DSS platform does not receive the message. How to troubleshoot that?

A: There are many factors that lead to this problem. We recommend investigating that from 2 perspectives.

Bridge: Increase the debug logs. You should first clarify whether the Bridge received the third-party events, and send the results to DSS platform.

DSS Platform: Check two configurations first. Firstly, you should check whether the event type and event configuration of the Bridge are consistent with the content of the message. Secondly, you should check whether the alarm scheme is configured and the **Notify User** is correctly selected. If there is nothing wrong with them, please contact the technical staff of DSS platform.

Q: Why is event-recording linkage configured, but no video is recorded when an event is triggered?

A: If the time zone of the server where the Bridge is located matches that of the server where the DSS-PRO is located, but there is a time inconsistency, this problem emerges. In light of this, we recommend installing Bridge directly on the server where DSS-PRO is located.

Q: How to verify whether the port is occupied?

A: Press the Windows + R key and enter the netstat -ano | find "port" in the pop-up command box. The port indicates the verified port. If the result is displayed, the port is occupied.

As shown in the following figure, port8000 is occupied.

```
C:\Users\357199>netstat -ano | find "8000"
TCP    127.0.0.1:8000      0.0.0.0:0          LISTENING       26200
```

Q: How to synchronize the changes in devices and people made on the third-party platform?

A: To synchronize any changes in devices and people made on the third-party platform, you can access the third-party docking page and click **Manual Collect** corresponding to the specific Project. This will update the data accordingly.

Dahua-Bridge-Development-and-Docking-Manual_V1.1.1_20240715 for HOTEL_BUDING